

Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists because everything after it assumes you know what a "terminal" is, what "running a script" means, and how to make sense of an error message. If you've never programmed before, read this first — it will make Day 1 dramatically less overwhelming. If any of this is already familiar, skim it and move on.

What is a computer program, really?

A computer only ever does one thing: it follows instructions, one at a time, incredibly fast. A "program" (also called "code" or a "script") is just a list of instructions written in a language the computer can understand.

Python is one such language — a way of writing instructions that a specific program (the "Python interpreter") reads and carries out, step by step, from top to bottom.

When people say "I wrote a Python script," they mean: they typed a list of instructions into a text file, and that file can be handed to the Python interpreter, which will read it and do exactly what it says — no more, no less, and never anything you didn't literally tell it to do. This is the single most important mental habit to build as a beginner: **the computer does exactly what your code says, not what you meant**. Almost every confusing bug in your first few weeks will come down to this gap.

What is a text editor, and what will you use?

Code is just text, written in a plain text file — not a Word document, no formatting, no fonts, just characters. You write it in a **text editor** built for code (as opposed to Notepad or Word). Popular ones: Visual Studio Code ("VS Code," free, extremely common), PyCharm, Sublime Text. If you don't already have one, install VS Code before continuing — you'll be living in it for the next two weeks.

A Python file is a plain text file whose name ends in `.py` — for example `hello.py`. That file extension is how your computer (and you) recognize it as Python code rather than, say, a text document.

What is "the terminal," and why does everyone use it?

The **terminal** (also called the "command line," "console," or "shell") is a text-based way of telling your computer to do things, instead of clicking icons. Instead of double-clicking a program's icon, you type its name and press Enter. It looks intimidating the first time because it's just a blank window with a blinking cursor and no buttons — but it's just a text conversation with your computer: you type a command, press Enter, and the computer prints back a response.

On Windows, you already have one open in this environment (PowerShell). You'll type commands like:

```
python --version
```

and press Enter, and the terminal will print something back, like `Python 3.14.5`. That's the whole interaction model: type a command, press Enter, read the output, repeat.

Why not just use icons and menus? Because almost every professional coding tool — running your code, installing packages, using version control (Day 13), talking to servers — is built to be operated from the terminal first, with graphical interfaces built on top later (if at all). Getting comfortable typing commands is a foundational skill, not a workaround.

Running a Python script

Once you've saved a file called `hello.py` containing:

```
print("Hello, world!")
```

you run it from the terminal by navigating to the folder that file is in, then typing:

```
python hello.py
```

and pressing Enter. Python reads `hello.py` top to bottom, executes each instruction, and in this case prints `Hello, world!` to the terminal. That's the entire cycle you'll repeat constantly for the next two weeks: **write code in a file** → **run it from the terminal** → **read the output** → **fix something** → **run it again**.

`print(...)` is a built-in instruction ("function," a term you'll fully learn on Day 4) that displays whatever is inside its parentheses on the screen. You will use `print()` constantly, especially early on, to see what your code is actually doing at each step — it's the single most useful tool for understanding your own code while you're learning.

What is the Python REPL (the "interactive shell")?

Instead of writing a whole file, you can also start Python directly in the terminal by typing just:

```
python
```

and pressing Enter. This drops you into the **REPL** (Read-Evaluate-Print Loop) — an interactive mode where you type one line of Python at a time, press Enter, and immediately see the result, without saving anything to a file first:

```
>>> 2 + 2
4
>>> print("hi")
hi
```

The `>>>` is Python's prompt, telling you it's waiting for input — you don't type the `>>>` yourself. The REPL is perfect for quickly testing a small idea ("what does this expression actually return?") without the overhead of creating a file. You'll be told throughout this course to "try it in the REPL" — this is what that means. To leave the REPL, type `exit()` and press Enter, or press Ctrl+Z then Enter on Windows.

Reading an error message without panicking

You will see error messages constantly — this is normal and does not mean you broke something badly. It means Python tried to follow your instructions and hit a step it couldn't do. Here's an example and how to read it:

```
print("Hello"
```

Running this (notice the missing closing parenthesis) produces:

```
File "hello.py", line 1
  print("Hello"
      ^
SyntaxError: '(' was never closed
```

Read error messages from the **bottom up**:

- 1 The last line (**SyntaxError: ...**) tells you the *category* of problem and a short description — here, an opening `(` that was never closed.
- 2 The line above with `^` points at roughly *where* Python got confused.
- 3 **File "hello.py", line 1** tells you *which file and line* to go look at.

The category name before the colon (**SyntaxError**, **NameError**, **TypeError**, **ValueError**, and many more) is itself useful information — you'll learn what several of these specifically mean starting Day 6, but even now, "SyntaxError" tells you the problem is how you *typed* the code (missing punctuation, bad structure), separate from "the logic is wrong" errors that only show up once the code actually runs.

The skill to build immediately: read the error message fully before doing anything else. New programmers often see red text, panic, and start randomly changing code. Instead: read what it says, find the line number it points to, and think about what that line is actually telling the computer to do.

Indentation — Python's way of showing "this belongs inside that"

Many languages use curly braces `{ }` to show which instructions belong together (e.g., "everything inside this block belongs to this **if**"). Python instead uses **indentation** — consistent leading whitespace — to mean the same thing. This isn't just a style preference; in Python, indentation is part of the actual syntax and changing it changes what the code means.

```
if 5 > 3:
    print("five is bigger")    # indented -- this line is INSIDE the if
    print("still inside")     # also indented -- also inside the if
print("this always runs")    # NOT indented -- outside the if, always runs
```

The convention (and what your editor will do automatically) is 4 spaces per indentation level — never tabs mixed with spaces, since Python will treat them inconsistently and raise an **IndentationError**. You'll internalize this within the first couple of days since VS Code and most editors indent automatically after a line ending in `:`.

Comments — notes to humans that Python ignores

Anything after a `#` on a line is a **comment** — Python skips it entirely; it's there purely for a human reading the code later (including future-you).

```
# This calculates the area of a rectangle
area = width * height    # width and height must already be defined above
```

Good comments explain *why* something is done a certain way, not *what* the code does (well-written code should make the "what" obvious just by reading it) — you'll see this convention followed throughout every

lesson in this course.

A few words you'll see constantly, defined up front

- **Variable** — a name that refers to a value (full explanation Day 1).
- **Function** — a named, reusable block of instructions you can "call" (run) by name, optionally handing it some input (full explanation Day 4). `print(...)` is a function.
- **Argument** — a value you hand to a function when you call it. In `print("hi")`, `"hi"` is the argument.
- **String** — text data, written between quotes, like `"hello"` or `'hello'` (either quote style works; be consistent).
- **Method** — a function that belongs to a specific piece of data, called with a dot, like `"hello".upper()`.
- **Object** — a piece of data in Python (a number, a string, a list, anything). In Python, genuinely everything is an object — you'll see exactly what this means and why it matters on Day 1.
- **Bug** — a mistake in your code that makes it behave incorrectly (or not run at all). Not a moral failing — every programmer, at every level, produces bugs constantly; the skill is finding and fixing them methodically, not avoiding them entirely.
- **Debugging** — the process of finding and fixing a bug (full toolset on Day 12, but you'll practice informally from Day 1 onward).

What "run the file and check PASS/FAIL" means for these exercises

Starting Day 1, every `exercises.py` file has spots marked `# TODO: implement`, followed by `pass` (a Python keyword meaning "do nothing," used as a placeholder so the file doesn't error while you're still working on it). Below that, the file has code that calls your function with test inputs and prints `PASS` or `FAIL` depending on whether your answer was correct. Your job each day:

- 1 Open `exercises.py` in your editor.
- 2 Replace `pass` under each `TODO` with real code that makes the function do what its docstring (the text in `"""triple quotes"""` right under the `def` line) describes.
- 3 Save the file, then run it from the terminal: `python exercises.py`.
- 4 Read the `PASS/FAIL` output. Fix anything that says `FAIL`, save, and run again.
- 5 Once everything passes, compare your solution to `solutions.py` in the same folder.

This loop — write, run, read output, fix, run again — is, in essence, the entire day-to-day job of a programmer at any experience level. You're not just learning Python syntax this week; you're building that loop into a habit.

You're ready for Day 1

Go to `day01-basics/lesson.md` next. Whenever a new term shows up that wasn't defined here, the lesson will define it right there, the first time it's used — you're not expected to know anything beyond what's in this file yet.

Day 1 - Variables, Types, and the Object Model

Day 1 — Variables, Types, and the Object Model

If you haven't read [day00-primer/lesson.md](#) yet, read that first — it covers what a terminal, a script, and the REPL are, which everything below assumes you already know.

Objectives

- Understand what a Python variable actually *is* — a name pointing at a value, not a labeled box holding it
- Meet the core built-in data types and understand what each one is *for*
- Understand the difference between two things that look similar but aren't: values that can change after creation ("mutable") and values that can't ("immutable"), and why this distinction causes real, common bugs
- Understand why Python lets a variable hold a number one moment and text the next ("dynamic typing")

What is a variable, really?

In everyday language, we might say "let `x` be 5" — and in most programming tutorials, that's illustrated as a labeled box: a box named `x`, with the number `5` inside it. That picture is close enough to get started with simple numbers, but it will actively mislead you once you meet lists and dictionaries later today — so let's replace it with the correct picture from the start.

Here's what actually happens when Python runs this line:

```
x = 5
```

- 1 Python creates an **object** in the computer's memory representing the value `5`. (An "object" is just Python's word for "a piece of data floating around in memory" — every single value in Python, from numbers to text to entire programs, is an object. You'll see why this matters more and more as the course goes on.)
- 2 Python creates a **name**, `x`, and points it at that object — like a sticky note with "x" written on it, stuck onto the `5` object.

That's it. A variable is a **name that points at an object** — not a container that holds a copy of the value. This distinction feels academic right now, with plain numbers, but watch what happens when two names point at the *same* object:

```
a = [1, 2, 3]    # a list -- more on lists later today
b = a           # b now points at the SAME object a points at -- not a copy!
a.append(4)      # this changes the object that a (and b) point to
print(b)         # [1, 2, 3, 4] <- surprising if you were picturing "boxes"!
```

If you were thinking of `a` and `b` as separate boxes, this result looks like a bug — `b` was never supposed to change, you never touched `b`! But once you picture `a` and `b` as two sticky notes stuck on the *same* physical list, the result makes total sense: there was only ever one list in memory. `a.append(4)` changed that one list, and both sticky notes ("names") see the same, single, changed object, because they were always pointing at the same thing.

This is going to come up constantly for the rest of this course, so let's build a tool to check it for ourselves.

Proving it with `id()`

Python has a built-in function called `id()` (a function is a named, reusable operation you can invoke by writing its name followed by parentheses — you'll learn to write your own on Day 4; for now you're just using ones Python already provides). `id(some_value)` returns a unique number identifying exactly which object in memory that value refers to — think of it as that object's "physical address" or serial number.

```
a = [1, 2, 3]
b = a
print(id(a))          # some number, e.g. 140185732791872
print(id(b))          # the EXACT SAME number -- proving a and b point at the same object
print(id(a) == id(b)) # True
```

Python gives you a shortcut for exactly this comparison: the `is` operator, which asks "are these two names pointing at the literal same object?" (as opposed to "do these two objects merely look equal?", which is what `==` asks):

```
print(a is b)         # True -- same object
c = [1, 2, 3]         # a brand NEW list that just happens to contain the same values
print(a is c)         # False -- two different objects in memory
print(a == c)         # True -- but their CONTENTS are equal
```

Keep `is` (identity: "the literal same object?") and `==` (equality: "same value/contents?") mentally separate — mixing them up is a classic beginner bug, especially once you learn about `None` on Day 2, where you'll be told to always write `x is None` rather than `x == None` (both usually work, but `is` is the semantically correct and universally-used idiom for this specific check).

Why does Python work this way? (the practical reason, not just trivia)

Imagine every single time you wrote `b = a`, Python secretly made a full, independent copy of whatever `a` pointed to. For a single number that's cheap and harmless. But what if `a` were a list with ten million items in it? Copying it on every single assignment, function call, and pass into another variable would make your programs slow and would use huge amounts of memory for no reason, most of the time. So Python's design instead just copies the *pointer* (the name-to-object link) — which is always cheap, no matter how big the underlying object is — and only makes an actual copy when *you* explicitly ask for one. You'll learn exactly how to ask for a copy later today (`.copy()`, `list(...)`, or slicing).

Mutable vs. immutable — the single most important distinction in this entire language

This is the concept that, if you truly understand it today, will save you hours of confusion in the coming weeks. Every type of value in Python falls into exactly one of these two categories:

- **Immutable** — once created, this object can *never* be changed. Any operation that looks like it's "modifying" it is secretly creating a brand new object instead. Types: `int` (whole numbers), `float` (decimal numbers), `bool` (True/False), `str` (text), `tuple` (a fixed, ordered group of values — introduced properly on Day 3), `frozenset`.

- **Mutable** — this object *can* be changed in place, after it's created, without creating a new object. Types: `list`, `dict`, `set` (all introduced Day 3), and any custom class you write yourself (Day 8) unless you deliberately design it not to be.

Watch this carefully:

```
x = 5
x = 6
```

It's tempting to describe this as "we changed `x` from 5 to 6." But that's not quite what happened, and the precise version matters: **integers are immutable — the object 5 was never changed (it can't be — nothing can change it)**. What actually happened: Python created a brand new object, 6, and moved the name `x` to point at that new object instead. The old 5 object still exists somewhere in memory for a moment, completely untouched, until Python notices nothing points at it anymore and cleans it up automatically (this automatic cleanup process is called "garbage collection" — you don't need to manage it yourself, just know that it happens).

Now compare with a mutable type:

```
s = "hello"
s[0] = "H"      # this raises: TypeError: 'str' object does not support item assignment
```

Strings are immutable — you cannot change a single character of an existing string object; Python won't let you, and raises an error the moment you try. If you want an uppercase version, you must create an *entirely new* string:

```
s = "hello"
s = "Hello"     # this doesn't modify the old string -- it creates a NEW string object
                # and points s at it instead. The old "hello" object is discarded.
```

Now let's see the mutable case, where in-place change genuinely is possible:

```
lst = [1, 2, 3]
lst.append(4)    # this DOES modify the existing list object, in place
print(lst)      # [1, 2, 3, 4] -- same object, now with different contents
```

`lst.append(4)` really does reach into the existing list object sitting in memory and add an item to it — no new object is created. This is the behavior that made `b` change when we only touched `a`, in the very first example of this lesson.

Why this matters when you pass values into functions

You haven't formally learned functions yet (that's Day 4), but you've likely already used `print(...)`, and you'll be writing your own simple ones very soon, so it's worth planting this seed now: when you hand a value into a function, Python copies the *name-to-object pointer* into the function, exactly like `b = a` above — never the underlying object itself. This means:

- If the function does something that mutates the object in place (like `.append(...)` on a list), the caller's variable sees that change too, because there was only ever one object.
- If the function just does `parameter_name = something_else` (pointing the *local* name at a different object), that has **zero** effect on the caller's variable — the caller's name still points at whatever it originally pointed at.

You'll see this demonstrated concretely, with real code you can run and predict yourself, in today's exercises ([mutate_vs_reassign_demo](#)) — and you'll return to this exact idea in much more depth on Day 4 once you're writing functions of your own.

The core built-in types — your basic vocabulary of data

Every piece of data in Python has a **type** — the type tells Python (and you) what *kind* of thing a value is, and therefore what you're allowed to do with it. You find out any value's type using the built-in `type()` function:

```
type(5)           # <class 'int'>
type(5.0)         # <class 'float'>
type("hi")       # <class 'str'>
type([1, 2])     # <class 'list'>
```

Type	Example	Mutable?	What it's for
int	42	No	Whole numbers, positive or negative, no decimal point. Python's integers have no size limit (unlike many languages) — you can have arbitrarily huge whole numbers without special handling.
float	3.14	No	Numbers with a decimal point. Internally stored in a format ("IEEE 754 double") that can't represent every decimal exactly, which is why <code>0.1 + 0.2</code> prints <code>0.30000000000000004</code> instead of exactly <code>0.3</code> — this is a well-known quirk of nearly every programming language, not a Python bug.

<code>bool</code>	<code>True</code>	No	Represents yes/no, true/false. Under the hood, <code>bool</code> is actually a special case of <code>int</code> — <code>True</code> behaves exactly like <code>1</code> and <code>False</code> like <code>0</code> in arithmetic (<code>True + True == 2</code> is actually true — try it in the REPL).
<code>str</code>	<code>"hi"</code>	No	Text ("string" is short for "string of characters"). Written between either single <code>'...'</code> or double <code>"..."</code> quotes — Python treats them identically, just be consistent within one file.
<code>list</code>	<code>[1, 2]</code>	Yes	An ordered, changeable collection of values, in square brackets. Full treatment Day 3.
<code>tuple</code>	<code>(1, 2)</code>	No	An ordered, UNchangeable collection — like a list that's locked once created. Full treatment Day 3.
<code>dict</code>	<code>{"a": 1}</code>	Yes	Short for "dictionary" — maps keys to values, like a real dictionary maps words to definitions. Full treatment Day 3.
<code>set</code>	<code>{1, 2}</code>	Yes	An unordered collection with no duplicate values allowed. Full treatment Day 3.

NoneType	None	—	None is Python's way of representing "no value" or "nothing here" — similar to what other languages call "null." There is exactly one None object in the whole running program; every variable set to None points at that same single object.
----------	------	---	---

You don't need to memorize this table — you'll use every one of these types so often over the next two weeks that they'll become second nature. Treat it as a reference to glance back at.

To check "is this value of type X," prefer `isinstance(value, X)` over comparing `type(value) == X` directly:

```
isinstance(5, int)      # True
isinstance(True, int)   # True -- bool is considered a "kind of" int, as explained above
```

`isinstance` is the idiomatic (meaning: the way experienced Python programmers actually write it) choice because, as you'll see on Day 9, it correctly handles categories of related types, not just one exact type — `type(x) == X` gets this subtly wrong in ways that don't matter yet, but will once you're writing your own classes.

Dynamic typing: a variable's type can change, because a variable is just a name

```
x = 5          # right now, the name x points at an int object
x = "hello"    # now x points at a completely different object -- a str
```

This is completely legal in Python and is called **dynamic typing**: a variable isn't locked to one type forever — it can point at any type of object, and which type it currently points to is only decided by whatever it's currently pointing at, checked while the program is actually running (as opposed to some languages, where you must declare "x is an integer, forever" up front, and the tool that turns your code into a runnable program will refuse to run it if you ever assign text to it later).

This flexibility is convenient, but it has a real cost: Python can't warn you in advance that you're about to misuse a variable (say, treating a string like a number) — that mistake will only surface when the misbehaving line actually runs, possibly deep inside a program that otherwise looked fine. This is a large part of **why** Day 12 (testing) matters so much professionally — tests are how Python programmers compensate for the lack of an upfront check that other languages get for free.

Truthiness — what counts as "true enough" in an `if`

You haven't learned `if` yet in full (that's Day 2, tomorrow) but since it's so central, here's a preview relevant to today's types: every single value in Python, of any type, has an implicit yes/no answer to "is this true-ish?"

when used somewhere expecting True/False, such as inside an `if`. The values that count as "false-ish" (called **falsy**) are, essentially, all the ways of representing "empty" or "nothing":

`False`, `None`, `0`, `0.0`, `""` (empty string), `[]` (empty list), `{}` (empty dict), and a few others you'll meet later (empty tuple, empty set). **Every other value is truthy** — including things that might surprise you at first, like the string `"False"` (a non-empty string containing the *word* "False" is still just a non-empty string, and is therefore truthy) or the number `-1` (nonzero, so truthy).

```
if []:
    print("this line never runs -- an empty list is falsy")
if [0]:
    print("this line DOES run -- a list containing one item (even the falsy number 0) is itself non"
```

Exercises

Open `exercises.py` in this same folder and implement each function where you see `# TODO`. Run the file from your terminal with `python exercises.py` and read the PASS/FAIL output. If anything says FAIL, that function isn't doing what its docstring describes yet — fix it and run again. Once everything passes, open `solutions.py` to compare your approach.

Day 2 - Control Flow: Conditionals, Loops, Comprehensions

Day 2 — Control Flow: Conditionals, Loops, Comprehensions

Objectives

- Understand `if/elif/else` — how your program makes decisions
- Understand that a `for` loop's real job is "go through each thing in this collection," not "count from one number to another" (counting is just the most common special case)
- Understand `while` loops, and when to reach for one instead of a `for` loop
- Understand `break`, `continue`, and the unusual `for...else`
- Write your first list/dict/set "comprehensions" — a compact way to build a new collection from an existing one
- Meet Python 3.10's `match` statement, a modern alternative to a long chain of `if/elif`

Why do programs need "control flow" at all?

So far (Day 1) every example ran every line, in order, exactly once, top to bottom. Real programs need to make decisions ("if the user is over 18, allow access; otherwise don't") and repeat work ("do this once for every item in this list of 500 orders"). **Control flow** is the umbrella term for the instructions that let your program skip lines, repeat lines, or choose between different lines to run, instead of always running everything in a straight line from top to bottom. `if`, `for`, and `while` are the three basic control flow tools you'll use constantly, starting today.

`if` / `elif` / `else` — making decisions

```
age = 20

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")
```

Read this exactly as English: "if `age` is greater than or equal to 18, print the first message; otherwise (`else`), print the second." The colon `:` at the end of the `if` line, and the indentation (from Day 0) on the lines below it, are what tell Python "these indented lines belong to this decision." Only one branch ever runs, never both.

When you have more than two possibilities, use `elif` (short for "else if") for each additional check, and Python checks them in order, top to bottom, stopping at the first one that's true:

```
score = 72

if score >= 90:
```

```

    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C" -- because 72 >= 70 was the first true condition it reached

```

Notice `elif` only gets checked if every condition *above* it was false — once `score >= 70` is found true, Python entirely skips the `else` and moves on past this whole block, without ever checking whether `score >= 90` twice or looking further.

Comparing values

The comparison operators you'll use inside `if` conditions: `==` (equal to — note: two equals signs, since a single `=` means "assign," as you learned Day 1), `!=` (not equal to), `>`, `<`, `>=`, `<=`. You combine multiple conditions with the words `and`, `or`, and `not`:

```

if age >= 18 and has_id:
    print("allowed in")

if is_weekend or is_holiday:
    print("no work today")

if not is_raining:
    print("no umbrella needed")

```

Python also lets you chain comparisons in a way that reads naturally and many other languages don't allow directly:

```

if 0 <= x < 10:
    print("x is between 0 and 9, inclusive-exclusive")

```

This is exactly equivalent to `0 <= x and x < 10`, just more compact and closer to how you'd write it in a math class.

The ternary — an `if` that produces a value

Sometimes you just want to pick between two *values*, not two whole blocks of instructions. Python has a compact one-line form for exactly this:

```
status = "adult" if age >= 18 else "minor"
```

Read it as: "the value is `'adult'` IF `age >= 18`, ELSE the value is `'minor'`." This is called a **ternary expression** (or "conditional expression"). Use it only when each branch is a single simple value — if either side needs more than one step of logic, use a full `if/else` block instead for readability.

`for` loops — repeating an action once per item

Here is the most important reframing in today's lesson: **a `for` loop's job is to walk through a collection of things, one at a time, running the same block of code for each one.** It does NOT fundamentally mean "count from number A to number B" — that's just the single most common thing people use it for, via a helper

called `range` (explained below).

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Read this as: "for each `fruit` found inside `fruits`, do the following." The loop runs its indented body once per item — three times here — each time with `fruit` pointing at the next item in the list, in order: first `"apple"`, then `"banana"`, then `"cherry"`. Once it's gone through every item, the loop ends automatically and the program continues after it.

You can loop over the individual characters of a string the exact same way, since a string is really just an ordered sequence of characters:

```
for character in "abc":
    print(character)    # prints a, then b, then c, on separate lines
```

`range()` — the tool for "count from A to B"

When you genuinely do want to count (run something exactly 5 times, or go from 1 to 100), Python provides `range(...)`, which produces a sequence of numbers to loop over:

```
for i in range(5):
    print(i)            # prints 0, 1, 2, 3, 4 -- five numbers, STARTING AT 0, and stopping BEFORE 5
```

This surprises almost every beginner at least once: `range(5)` produces `0, 1, 2, 3, 4` — five numbers total, but stopping *before* reaching 5, not including it. This "starts at 0, stops just before the number you gave it" behavior is consistent across many parts of Python (you'll see it again with list slicing on Day 3), so it's worth internalizing now rather than re-deriving it every time.

`range` also accepts a start and a step: `range(2, 10, 2)` produces `2, 4, 6, 8` (starts at 2, stops before 10, counting by 2s each time).

One more thing worth knowing, even though it won't change how you use `range` day to day:

`range(10_000_000)` does *not* actually create ten million numbers in memory all at once — it's a special, lightweight object that produces each number on demand, only as the loop asks for the next one. This matters for performance on huge ranges, and it's your very first preview of an idea called "laziness" that you'll meet properly on Day 10.

Looping over a dictionary

You'll properly meet dictionaries on Day 3, but since you'll see this pattern constantly, here's a preview: looping over a dict by default gives you its keys, one at a time, not its values:

```
scores = {"Ana": 90, "Bo": 85}

for name in scores:
    print(name)          # prints "Ana", then "Bo" -- the KEYS

for name, score in scores.items():
    print(name, score)    # prints "Ana 90", then "Bo 85" -- BOTH key and value, via .items()
```

`enumerate()` — when you need the position AND the item

If you need to know *where* you are in a list while also having the item itself, resist the beginner instinct to write `for i in range(len(fruits))` and then index in with `fruits[i]` — Python has a cleaner, more idiomatic built-in for exactly this:

```
for index, fruit in enumerate(fruits):
    print(index, fruit)    # 0 apple / 1 banana / 2 cherry
```

`zip()` — walking two lists together

If you have two related lists and want to process them in lockstep (item 1 from each together, then item 2 from each together, and so on), `zip` does this for you, stopping automatically once the shorter list runs out:

```
names = ["Ana", "Bo"]
ages = [30, 25]

for name, age in zip(names, ages):
    print(name, age)    # "Ana 30" then "Bo 25"
```

`while` loops — repeat until a condition becomes false

A `for` loop is for "do this once per item in a known collection." A `while` loop is for "keep doing this for as long as some condition stays true" — useful when you don't know in advance how many times you'll need to repeat something.

```
count = 0
while count < 3:
    print("still going:", count)
    count += 1    # shorthand for: count = count + 1
print("done")
```

This prints "still going: 0", "still going: 1", "still going: 2", then stops, because once `count` becomes 3, the condition `count < 3` becomes false and the loop ends. **If you forget to update `count` inside the loop, this becomes an infinite loop** — the condition never becomes false, and the program will run forever (or until you stop it with Ctrl+C in the terminal). This is an extremely common beginner mistake; if your program seems to hang and never finish, an infinite loop is the first thing to suspect.

A rough rule of thumb for choosing between them: **if you know exactly what collection you're going through, use `for`**. **If you're waiting for some condition to change (user input, a value crossing a threshold, "keep trying until it works"), use `while`**.

`break` and `continue` — early exits from a loop

`break` immediately stops the loop entirely, skipping any remaining items:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        break    # stop the loop the instant we hit 3
    print(number)    # prints only 1, then 2
```

`continue` skips just the *rest of the current pass* through the loop body and moves on to the next item, without stopping the whole loop:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        continue
```

```

        continue          # skip printing 3 specifically, but keep looping
    print(number)         # prints 1, 2, 4, 5 -- everything except 3

```

The unusual `for...else`

Python has an `else` clause you can attach to a loop, which is genuinely unusual among programming languages and confuses even people with experience in other languages the first time they see it. It runs only if the loop finished normally, *without* being stopped early by a `break`:

```

numbers = [1, 3, 5, 7]

for n in numbers:
    if n % 2 == 0:          # % is the "modulo" operator -- remainder after division
        print("found an even number:", n)
        break
else:
    print("no even numbers found") # this runs, since the loop never hit `break`

```

This is most useful for exactly this "search for something, and if you never find it, do X" pattern. It's used somewhat rarely in real code because many people find it non-obvious to read — but you should be able to recognize it when you encounter it in someone else's code.

Comprehensions — a compact way to build a new list from an old one

A **list comprehension** is not a new kind of loop — it's a compact expression that builds a brand new list by describing what you want to do to each item of an existing collection, all on one line. It's easiest to learn by comparing the long way and the short way side by side:

```

# the long way, using a full for loop
squares = []
for n in range(10):
    squares.append(n * n)

# the exact same result, as a list comprehension
squares = [n * n for n in range(10)]

```

Read a comprehension **right to left**, roughly as: "for each `n` in `range(10)`, give me `n * n`, and collect all those results into a new list." Both versions above produce the identical list `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]` — the comprehension is just a shorter way to write the same idea once it's familiar.

You can also filter which items are included, by adding an `if` at the end:

```

evens = [n for n in range(20) if n % 2 == 0] # only keep n where n % 2 == 0 (no remainder -- means even)

```

And you can transform values conditionally, using the ternary from earlier in this lesson, inside the expression part:

```

labels = ["even" if n % 2 == 0 else "odd" for n in range(5)]
# ["even", "odd", "even", "odd", "even"]

```

Dict comprehensions and set comprehensions work identically, just with `{}` instead of `[]`, and, for dicts, a **key: value** pair instead of a single expression — you'll use these more once you've properly met dicts and sets tomorrow (Day 3), but here's a preview:


```
squares_map = {n: n * n for n in range(5)}    # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

A word of caution, as a beginner: comprehensions are popular in Python because experienced programmers find them fast to read once they're used to the pattern — but as a beginner, if a comprehension is starting to feel like a puzzle you have to squint at, **write it as a normal, multi-line `for` loop instead**. A correct, readable loop beats a clever one-liner you can't quite explain. You'll develop a feel for when a comprehension is genuinely clearer over the next two weeks; there's no rush to force it before then.

`match` — Python's modern multi-way branch (Python 3.10+)

For situations with many possible exact values to check against (similar to a long chain of `elif`, but often clearer to read), Python 3.10 introduced `match`:

```
def describe_day(day_number):
    match day_number:
        case 1:
            return "Monday"
        case 2:
            return "Tuesday"
        case 6 | 7:
            return "Weekend!"
        case _:
            return "Unknown day"
```

the | here means "either of these values"

`\_` is the catch-all, like "else" -- matches anything

`match` can do more than this simple example shows (it can pull apart, or "destructure," more complex data like tuples — you'll see that in the exercises below), but for now, think of it as a cleaner alternative to a long `if/elif/elif/.../else` chain when you're comparing one value against several specific possibilities.

Exercises

Open `exercises.py`, implement each function at the `# TODO` markers, and run it with `python exercises.py` to check your work against the PASS/FAIL output.

Day 3 - Data Structures: Lists, Tuples, Dicts, Sets

Day 3 — Data Structures: Lists, Tuples, Dicts, Sets

Objectives

- Build real intuition for when to reach for each of the four core containers, in plain terms
- Understand, at a level you could explain to someone else, **why** checking membership in a dict/set is so much faster than in a list
- Get comfortable with slicing — a piece of syntax you'll use constantly from here on
- Learn the list/dict/set methods (a "method" is a function attached to a specific piece of data, called with a dot, like `my_list.append(...)`) you'll reach for daily

Why do we need more than one kind of container?

Yesterday you used lists briefly. Today you'll meet the full family of four built-in "containers" — ways of holding multiple values in one variable — and learn that each one exists because it makes a different trade-off. Choosing the right one isn't just a style preference; it affects how fast your code runs and how clearly it communicates your intent to whoever reads it later (including future-you).

Container	Ordered?	Allows duplicates?	Changeable after creation?	How you look things up	What it's for
<code>list</code>	yes	yes	yes	by position (index)	An ordered sequence of items you expect to add to, remove from, or reorder.
<code>tuple</code>	yes	yes	no	by position (index)	A fixed-size group of values that will never change shape — e.g., "this is always exactly an (x, y) pair."

dict	yes (Python remembers insertion order since version 3.7)	keys: no / values: yes	yes	by key (a name you choose), not position	"Look this up by name," like a real dictionary maps a word to its definition.
set	no	no (automatically removes duplicates)	yes	only "is this in here?" — no positional lookup at all	Fast "have I already seen this?" checks, removing duplicates, and comparing two groups of items.

Don't try to memorize this table right now — the rest of today's lesson explains each row in detail, with examples, and by the end it'll feel natural.

Lists — your everyday, general-purpose container

A list is written with square brackets, values separated by commas:

```
fruits = ["apple", "banana", "cherry"]
```

You already used lists briefly on Day 1 and Day 2. Lists are **ordered** (the items stay in the sequence you put them in) and **mutable** (you can change, add, and remove items after creating the list).

Getting one item out — indexing

Every item in a list has a position number, called an **index**, starting from **0** (not 1 — this trips up nearly every beginner at least once, so say it out loud: the first item is at index 0):

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])    # "apple" -- the FIRST item
print(fruits[1])    # "banana"
print(fruits[2])    # "cherry"
print(fruits[-1])   # "cherry" -- negative indices count from the END; -1 is the LAST item
print(fruits[-2])   # "banana" -- second-to-last
```

Asking for an index that doesn't exist raises an error:

```
print(fruits[10])    # IndexError: list index out of range
```

This is one of the most common errors you'll see in your first weeks — it means you asked for a position that doesn't exist in that particular list, usually because the list was shorter than you assumed, or because of the classic off-by-one mistake (using **1** when you meant **0**, or going one item too far).

Slicing — getting a *range* of items at once

Slicing lets you pull out a sub-section of a list (or a string!) using the pattern `sequence[start:stop:step]`. All three parts are optional, and there's one rule to memorize: **start is included, stop is NOT included** (this is the exact same "stops just before the number you gave it" behavior you saw with `range()` yesterday — Python is consistent about this on purpose).

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

numbers[2:5]      # [2, 3, 4]          -- start at index 2, stop BEFORE index 5
numbers[:3]       # [0, 1, 2]         -- no start given, so it defaults to the beginning
numbers[7:]       # [7, 8, 9]         -- no stop given, so it defaults to the end
numbers[::2]      # [0, 2, 4, 6, 8]   -- no start/stop, step of 2 means "every other item"
numbers[::-1]     # [9, 8, 7, ..., 0] -- a step of -1 walks backward, reversing the list
numbers[-3:]      # [7, 8, 9]         -- negative indices work in slices too: "the last 3 items"
```

Important beginner gotcha: **slicing always returns a brand new list**, completely separate from the original — unlike plain indexing (`numbers[2]`), which just hands you the single item that was already there. This means `numbers[:]` (an empty slice, meaning "everything") is a common, quick way to make an independent copy of an entire list.

The list methods you'll use constantly

A "method" is just a function that belongs to a particular value, called by writing the value, then a dot, then the method name and parentheses:

```
groceries = ["milk", "eggs"]

groceries.append("bread")    # adds "bread" to the END of the list
print(groceries)             # ["milk", "eggs", "bread"]

groceries.insert(0, "coffee") # inserts "coffee" at index 0, pushing everything else along
print(groceries)             # ["coffee", "milk", "eggs", "bread"]

last_item = groceries.pop()   # removes AND returns the last item
print(last_item)              # "bread"
print(groceries)              # ["coffee", "milk", "eggs"]

groceries.remove("milk")      # removes the first item that EQUALS "milk" (by value, not position)
print(groceries)             # ["coffee", "eggs"]

groceries.sort()              # sorts the list IN PLACE (modifies it directly)
print(groceries)              # ["coffee", "eggs"] (alphabetical here)

numbers = [3, 1, 2]
newly_sorted = sorted(numbers) # returns a NEW sorted list, leaves `numbers` unchanged
print(numbers)                 # [3, 1, 2] -- untouched
print(newly_sorted)            # [1, 2, 3]
```

A very common beginner bug: writing `result = my_list.sort()` and then being confused that `result` is `None`. This happens because `.sort()` is designed to modify the list *in place* and, as a signal of that, deliberately returns nothing (`None`) rather than the list itself — its job is to change `my_list` directly, not to hand you a new value. Whenever you want the sorted result as a usable value without touching the original list, use `sorted(my_list)` instead, which returns a new list and leaves the original alone.

`len(some_list)` (a built-in function, not a method) tells you how many items are in a list — you'll use this constantly:

```
print(len(groceries))    # 2
```

Dictionaries — looking things up by name instead of position

A **dictionary** (or "dict" for short) stores **key-value pairs** — every value is associated with a specific key you choose, and you look values up by that key, not by position. Think of a real paper dictionary: you don't say "give me word #4,281" — you look up a specific *word* and get back its definition. Python dicts work the same way, and the "word" (called the **key**) doesn't have to be text — though it very often is.

```
scores = {"Ana": 90, "Bo": 85, "Cy": 78}

print(scores["Ana"])      # 90 -- look up by KEY, not position
scores["Dee"] = 95        # add a brand new key-value pair
scores["Ana"] = 92        # UPDATE an existing key's value
print(scores)             # {"Ana": 92, "Bo": 85, "Cy": 78, "Dee": 95}
```

Asking for a key that doesn't exist raises an error:

```
print(scores["Zed"])      # KeyError: 'Zed'
```

This is dicts' equivalent of a list's **IndexError** — it means "that key isn't in this dictionary." Since it's extremely common to want "give me the value if this key exists, otherwise give me some default, and don't crash," dicts have a built-in method for exactly that:

```
print(scores.get("Zed"))   # None -- no error, just "nothing found"
print(scores.get("Zed", 0)) # 0 -- your own chosen default instead of None
```

Looping over a dict, as previewed yesterday:

```
for key in scores:         # loops over KEYS by default
    print(key)

for key, value in scores.items(): # .items() gives you BOTH, as pairs
    print(key, value)

for value in scores.values():    # just the values, if you don't need the keys
    print(value)
```

Why is looking something up in a dict so much faster than in a list?

This is a genuinely important idea, and one that interviewers for programming jobs like to ask about, so it's worth understanding, not just memorizing. If you want to check whether "Ana" is a key in a dict with a million entries, Python does **not** have to check all one million keys one by one. Here's why:

A dict is built on something called a **hash table**. When you store a key, Python runs it through a function called **hash()**, which turns the key into a number (imagine it as a fingerprint unique to that value). That number tells Python exactly which "slot" in an internal table to put the entry into. Later, when you look up that same key, Python re-computes its hash and jumps *directly* to that slot — no scanning required, regardless of how many other entries exist. This lookup is, on average, done in "constant time" (programmers write this as **O(1)** — you'll see this notation again) — meaning it takes roughly the same tiny amount of time whether the dict has 10 entries or 10 million.

Contrast this with a list: to check whether a value is *anywhere* in a list, Python has no shortcut — it has to check the first item, then the second, then the third, and so on, until it either finds a match or reaches the end. This is called **O(n)** ("linear time" — the time grows in proportion to how many items there are, *n*). For a small list this difference is invisible; for a large one, it's the difference between instant and noticeably slow.

This is also exactly why dict keys (and set items, below) must be immutable types (numbers, strings, tuples — the immutable types from Day 1), and why you'll get an error if you try to use a list as a key:

```
d = {[1, 2]: "value"}    # TypeError: unhashable type: 'list'
d = {(1, 2): "value"}    # fine -- tuples ARE hashable (since they can't be changed after creation)
```

If Python allowed a mutable list as a key, and you later changed that list, its "fingerprint" (hash) would change too — and the dict would no longer be able to find the slot it originally filed that entry under. Rather than allow a subtle, hard-to-track bug like that, Python simply forbids mutable types as dict keys entirely.

Sets — fast membership checks and "no duplicates allowed"

A **set** is an unordered collection that automatically throws away duplicate values and, like a dict's keys, is backed by that same hash-table trick — so checking "is this item in the set?" is just as fast as a dict lookup, for the same underlying reason.

```
unique_visitors = {"ana", "bo", "ana", "cy"}
print(unique_visitors)    # {"ana", "bo", "cy"} -- the duplicate "ana" was automatically dropped
print("bo" in unique_visitors)    # True -- fast membership check
```

Sets support the same operations you might remember from a math class's "set theory" unit:

```
a = {1, 2, 3}
b = {2, 3, 4}

a | b    # union: everything in EITHER set -> {1, 2, 3, 4}
a & b    # intersection: only what's in BOTH -> {2, 3}
a - b    # difference: in a but NOT in b -> {1}
a ^ b    # symmetric difference: in exactly one of them, not both -> {1, 4}
```

A very practical real-world use: if you have two lists and want to know what they have in common, converting both to sets and using `&` is dramatically faster than writing nested loops to compare every item in one list against every item in the other, especially as the lists grow.

Tuples — a "list" that's locked once created

A tuple looks almost like a list but uses parentheses instead of square brackets, and, crucially, **cannot be changed after it's created** — no appending, no removing, no reassigning an item at a position.

```
point = (3, 4)
point[0] = 99    # TypeError: 'tuple' object does not support item assignment
```

Since a tuple can never change shape, using one is a signal to yourself and anyone reading your code: **"this is a fixed-size record, not a growable collection."** A very common place you'll encounter tuples without necessarily reaching for them on purpose: a function that needs to hand back more than one value actually returns a tuple behind the scenes.

```
def min_and_max(numbers):  
    return min(numbers), max(numbers)  # this is really returning the tuple (min(numbers), max(numbers))  
  
lowest, highest = min_and_max([3, 1, 4, 1, 5])  # "unpacking" the tuple into two separate names  
print(lowest, highest)                          # 1 5
```

That last line — assigning `lowest, highest = ...` — is called **unpacking**: Python takes a tuple (or list) with exactly as many items as you have names on the left, and assigns them in order. You'll use this pattern constantly once you start writing your own functions on Day 4.

Exercises

Open `exercises.py`, fill in each `# TODO`, and run `python exercises.py` to see PASS/FAIL for each one.

Day 4 - Functions: Args, Scope, Closures, First-Class Functions

Day 4 — Functions: Args, Scope, Closures, First-Class Functions

Objectives

- Understand what a function actually is and why they're one of the most important tools in programming
- Write functions that accept required parameters, optional parameters with defaults, and flexible numbers of arguments
- Understand "scope" — which parts of your program can see which variables, and why
- Understand closures — functions that remember values from where they were created
- Understand that functions themselves are just another kind of value in Python

What is a function, and why do we need them?

You've already been *using* functions since Day 0 — every time you wrote `print(...)`, `len(...)`, or `range(...)`, you were calling a function that Python already built for you. A **function** is a named, reusable block of instructions. Instead of copy-pasting the same five lines of code every time you need to do something, you write those five lines once, inside a function, give it a name, and then just write that name (followed by parentheses) every time you want to run those instructions again — optionally handing it different input each time.

Here's how you define your own:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Ana")    # prints: Hello, Ana!  
greet("Bo")     # prints: Hello, Bo!
```

Let's name every part of this, since the vocabulary matters for reading documentation and error messages later:

- `def` is the keyword that starts a function definition.
- `greet` is the function's **name** — you choose this, just like a variable name.
- `name` (inside the parentheses) is a **parameter** — a placeholder for a value the function expects to be given each time it's used. Parameters are only defined here, in the function's own definition.
- `"Ana"` and `"Bo"`, when you actually call the function, are **arguments** — the real, specific values you hand in for each parameter, for that particular call.
- The indented lines below `def greet(name):` are the function's **body** — the instructions it runs each time it's called. Exactly like `if` and `for`, indentation is what tells Python which lines belong inside the function.

- `f"Hello, {name}!"` is an **f-string** (formatted string) — the `f` right before the opening quote lets you drop a variable's value directly into the middle of text using `{curly braces}`. You'll use these constantly; full details on Day 5.

"Calling" a function just means writing its name followed by parentheses (with any arguments inside), which tells Python "go run that function's body right now, using these specific argument values."

``return`` — sending a value back out of a function

`print(...)` above just displays something and gives nothing back to use later. Often, you want a function to **compute** something and hand the result back to whatever called it, so you can store it in a variable or use it in further calculations. That's what `return` does:

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)    # 7
```

The moment Python hits a `return` statement, it immediately stops running the function and hands that value back to wherever the function was called from. Any code written after a `return` inside the same block will never run — this is worth knowing because it explains a certain category of "why isn't this line doing anything?" bug.

If a function never hits a `return` statement (like `greet` above, which only `prints`), calling it still technically produces a value — Python automatically gives you back `None` (the "nothing here" value from Day 1) if you try to use the result of a function that never explicitly returned anything:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
output = greet("Ana")    # this DOES print "Hello, Ana!" (the print() call still runs)  
print(output)            # None -- greet() never used `return`, so its "result" is None
```

Default parameter values — making an argument optional

You can give a parameter a default value, which is used automatically if the caller doesn't provide their own:

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Ana")                # Hello, Ana!           -- greeting uses its default  
greet("Bo", "Good morning") # Good morning, Bo!    -- greeting overridden explicitly
```

Any parameter with a default value **must come after** all parameters without one — Python requires this ordering so it can always figure out, unambiguously, which argument belongs to which parameter.

A genuinely dangerous trap: mutable default values

This next part is one of the most commonly-cited "gotchas" in all of Python, and it's worth understanding **why** it happens, not just memorizing the fix, because the underlying reason connects directly back to Day 1's mutability lesson.

```
def add_item(item, bucket=[]):    # DANGER -- do not do this
```

```

    bucket.append(item)
    return bucket

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['a', 'b'] <- probably NOT what you expected! You'd expect ['b']

```

Why does this happen? Default values are created exactly **once** — at the moment Python reads the `def` line and creates the function — not fresh on every single call. So `bucket=[]` creates exactly one list object, one single time, and every call that doesn't supply its own `bucket` argument reuses that **same** list. Because lists are mutable (Day 1), `.append(...)` from the first call permanently changes that one shared list, and the second call sees the leftover change from the first.

The fix, which you should treat as a standard pattern to reach for automatically whenever you need a mutable default:

```

def add_item(item, bucket=None):
    if bucket is None:                # note: `is None`, not `== None` -- the idiomatic way to check
        bucket = []                  # a BRAND NEW empty list, created fresh on THIS call only
    bucket.append(item)
    return bucket

```

Now every call that doesn't supply its own bucket gets its own fresh, independent list, created new each time the function actually runs.

`\*args` and `\*\*kwargs` — accepting a flexible number of arguments

Sometimes you don't know in advance exactly how many arguments a function will be called with. `*args` collects any extra positional arguments (arguments given without a name) into a tuple; `**kwargs` collects any extra keyword arguments (arguments given as `name=value`) into a dict. (`args` and `kwargs` are just conventional names — the `*` and `**` are what actually matter.)

```

def describe(name, *tags, active=True, **extra):
    print(name, tags, active, extra)

describe("server1", "prod", "web", region="us-east", owner="ana")
# prints: server1 ('prod', 'web') True {'region': 'us-east', 'owner': 'ana'}

```

Here, `"server1"` fills the required `name` parameter; `"prod"` and `"web"` (extra values with no name attached) are swept up into `tags` as a tuple; `region="us-east"` and `owner="ana"` (extra named values) are swept up into `extra` as a dict. You'll see `*args`, `**kwargs` constantly in real-world code, especially any time one function needs to "forward" whatever it was given along to another function without needing to know its exact signature in advance — this becomes especially important once you meet decorators on Day 10.

Scope — which parts of your code can "see" which variables

Scope refers to which parts of your program a given variable name is visible from. A variable created **inside** a function only exists inside that function — it's invisible to code outside it, and it's created fresh, then thrown away, every single time the function is called:

```
def calculate():
    result = 42
    return result

calculate()
print(result)    # NameError: name 'result' is not defined
```

`result` only exists during the time `calculate()` is actually running — once it returns, `result` is gone. This is genuinely useful: it means the internal, working variables of one function can never accidentally clash with the internal variables of another function, even if they happen to share the same name.

Python looks up a name using a specific search order, often abbreviated **LEGB**: **L**ocal (inside the current function) → **E**nclosing (any function this one is nested inside) → **G**lobal (the top level of your file) → **B**uilt-in (Python's own built-in names, like `print` or `len`). Python checks each level in that order and uses the first match it finds.

```
message = "global"           # a GLOBAL variable -- lives at the top level of the file

def show_message():
    message = "local"        # a LOCAL variable -- only exists inside show_message
    print(message)           # "local" -- found immediately in the Local scope, search stops there

show_message()
print(message)               # "global" -- unaffected; the function's `message` was a totally sep
```

`global` — explicitly modifying a variable from outside a function

By default, if you try to *assign* to a variable inside a function, Python assumes you mean a brand new local variable — even if a global variable of the same name already exists — which leads to a specific, common error:

```
count = 0

def increment():
    count = count + 1    # UnboundLocalError!

increment()
```

This fails because Python sees you assigning to `count` somewhere inside `increment`, decides `count` must be local to this function, and then — reading left to right — tries to read `count`'s current value before it's ever been given one locally. To genuinely modify the outer, global `count`, you must say so explicitly with the `global` keyword:

```
count = 0

def increment():
    global count
    count = count + 1

increment()
print(count)    # 1
```

In real, professional code, relying on `global` is usually considered a sign to reconsider your design — passing values in as parameters and getting results back via `return` (as you did earlier in this lesson) is almost always clearer and less error-prone than quietly mutating some shared global variable from inside a function. You're learning `global` today so you can recognize it and understand *why* the error happens without it — not

because you should reach for it often.

Closures — a function that remembers where it came from

This is one of the more mind-bending ideas in today's lesson the first time you see it, so don't worry if it takes a second read. A function defined *inside* another function can "remember" the variables from the outer function, even after the outer function has already finished running:

```
def make_multiplier(factor):
    def multiply(n):
        return n * factor    # `factor` comes from the ENCLOSING function, not from `multiply` itself
    return multiply          # we return the INNER function itself, not the result of calling it

double = make_multiplier(2)    # factor is 2 for this particular "version" of multiply
triple = make_multiplier(3)    # factor is 3 for THIS version -- entirely separate from double's

print(double(5))    # 10
print(triple(5))    # 15
```

`multiply` is called a **closure**: it "closes over" the variable `factor` from the function it was created inside, and keeps its own private copy of that value alive for as long as the closure itself exists — even though `make_multiplier` finished running and returned long ago. Each separate call to `make_multiplier` creates a completely independent `factor`, which is why `double` and `triple` don't interfere with each other at all, despite being built from the exact same inner function definition.

You'll rely on this exact mechanism to understand decorators on Day 10 — it's worth re-reading this section then if it doesn't fully click yet today.

Functions are values too — you can pass them around like any other data

This is a genuinely important idea: in Python, a function is just another kind of object (remember, from Day 1: everything in Python is an object) — which means you can store it in a variable, put it inside a list or dict, and pass it into other functions as an argument, exactly like a number or a string.

```
def square(x):
    return x * x

operation = square    # note: NO parentheses -- we're referring to the function ITSELF, not calling it
print(operation(5))    # 25 -- calling it through the new name works identically

def apply_twice(func, value):
    return func(func(value))    # calls whatever function was passed in, twice in a row

print(apply_twice(square, 3))    # 81 (square(3) is 9, then square(9) is 81)
```

`lambda` gives you a way to create a small, unnamed ("anonymous") function inline, useful for quick one-off cases, most often as an argument to another function:

```
words = ["banana", "kiwi", "fig"]
words.sort(key=lambda word: len(word))    # sort by each word's LENGTH instead of alphabetically
print(words)    # ['fig', 'kiwi', 'banana']
```

`lambda word: len(word)` is a tiny, unnamed function equivalent to `def get_length(word): return len(word)`. Reach for `lambda` only when the logic is genuinely a single simple expression — anything more involved should be a normal, named `def` function instead, which is easier to read and to debug.

Exercises

Open `exercises.py`, fill in each `# TODO`, then run `python exercises.py` and read the PASS/FAIL results.

Day 5 - Strings, Formatting, Files, and Context Managers

Day 5 — Strings, Formatting, Files, and Context Managers

Objectives

- Get comfortable with the string methods you'll use daily
- Write f-strings fluently, including controlling how numbers are displayed
- Read and write real files on disk, correctly
- Understand *why* the `with` keyword exists, not just that you should type it

Strings are immutable — a quick reminder, now with the methods that matter

Day 1 established that strings can't be changed in place — every string "modification" you do actually produces a brand new string, leaving the original completely untouched. Keep that in mind as you read the methods below: every single one of them **returns a new string** rather than changing the one you called it on.

```
s = " Hello, World! "
```



```
s.strip()          # "Hello, World!"      -- removes whitespace from both ends
s.lower()          # " hello, world! "    -- an all-lowercase COPY
s.upper()          # "  HELLO, WORLD!  "
s.replace("l", "L") # "  HeLlo, WorLd! "  -- replaces every occurrence
s.split(",")        # [' Hello', ' World! '] -- splits into a list wherever "," appears
",".join(["a", "b", "c"]) # "a,b,c" -- the REVERSE of split: glue a list of strings together
s.startswith(" He") # True
s.endswith("! ")    # True
"42".isdigit()      # True -- checks whether every character is a digit
s.find("World")      # 9 -- the index where "World" starts, or -1 if not found
len(s)              # the number of characters, including the spaces
```

Notice that none of these change `s` itself — if you want to keep the result, you must assign it, usually back to the same name:

```
s = " Hello, World! "
s = s.strip() # NOW s actually refers to the new, trimmed string
```

`.split()` — a common beginner trap

Calling `.split()` with **no argument at all** splits on *any run* of whitespace (spaces, tabs, newlines) and automatically discards empty results:

```
" hi there ".split() # ['hi', 'there']
```

But calling `.split(" ")`, giving it an explicit single space, splits on that *exact* literal character every time it appears — including when that produces empty strings from doubled-up spaces:

```
" hi there ".split(" ")      # ['', ' ', 'hi', ' ', ' ', 'there', ' ', '']
```

These look similar but behave very differently — this exact mismatch is a genuinely common source of bugs when parsing text that might have inconsistent spacing (like typed-in user input, or lines from a file). When in doubt, prefer plain `.split()` with no argument unless you specifically need to split on one exact character (like a comma in a CSV line, which you'll see on Day 11).

f-strings — the modern way to build text that includes variable values

An **f-string** lets you embed the value of a variable (or any expression) directly inside a piece of text, by writing **f** right before the opening quote and wrapping the variable in **{curly braces}**:

```
name = "Ana"
score = 91.5

print(f"{name} scored {score}%")          # "Ana scored 91.5%"
print(f"{name} scored {score:.1f}%")      # "Ana scored 91.5%" -- .1f means "1 digit after the d
print(f"{score=}")                        # "score=91.5" -- shows the variable name AND its v
```

The part after a colon inside the braces (like `.1f` above) is called a **format spec** — it controls exactly how the value is displayed:

```
f"{3.14159:.2f}"          # "3.14" -- round to 2 decimal places
f"{1000000:,}"           # "1,000,000" -- insert thousands separators
f"{0.5:.0%}"             # "50%" -- format as a percentage
f"{ 'left':<10}|"         # "left      |" -- left-align within a width of 10 characters
f"{ 'right':>10}|"        # "          right|" -- right-align within a width of 10
```

You'll use f-strings constantly, in essentially every piece of Python code you write from here on — for printing results, building error messages, and formatting output for a person to read. They're the modern, preferred way to do this in Python; you may encounter two older styles in existing code you read (**%-formatting** and `.format()`) — recognize them if you see them, but write new code with f-strings.

Files — reading and writing data that outlives your program

Everything you've done so far has lived only in your program's memory while it runs, and vanished the moment it finished. A **file** lets your program save data permanently on disk, so it's still there the next time you run the program (or open it in another program entirely). Opening a file gives you a **file object** — a temporary handle Python uses to read from or write to that file.

```
with open("data.txt", "r") as f:
    contents = f.read()
print(contents)
```

`open("data.txt", "r")` opens the file named `data.txt` in **read mode** (`"r"`, the default if you don't specify a mode) and gives you back a file object, which this code names `f`. `f.read()` reads the *entire* file's contents into one big string.

Why `with`? — the problem it solves

Opening a file means your program is holding onto a limited resource that the operating system is tracking (only so many files can be open by a program at once, and other programs may be waiting to use the same file). You're supposed to **close** a file once you're done with it, to release that resource:

```
f = open("data.txt")
data = f.read()
f.close()          # if anything above this line raises an error, close() NEVER RUNS
```

The problem: if something goes wrong between `open()` and `close()` — say, `f.read()` raises an unexpected error — `f.close()` is skipped entirely, and the file is left open longer than it should be. Doing this by hand, reliably, in every possible situation (including ones you didn't anticipate) is genuinely hard to get right.

`with` solves this completely. It's called a **context manager**, and its entire job is: "run this setup, run my code, and no matter what happens — even if an error occurs partway through — guarantee the cleanup step runs afterward."

```
with open("data.txt") as f:
    data = f.read()
# by this point, f is GUARANTEED to be closed -- whether the block above succeeded or raised an error
```

The rule to internalize starting today: any time you open a file, always use `with`. You'll learn the mechanism behind exactly how `with` guarantees this (two special methods called `__enter__` and `__exit__`) on Day 9, once you've learned how to write your own classes — for now, just know that `with` is the correct, safe way to work with files, full stop.

Reading a file, a few different ways

```
with open("data.txt") as f:
    for line in f:          # loop directly over the file object -- gives you one line at a time
        print(line.strip()) # .strip() removes the trailing newline character each line ends w/ \n

with open("data.txt") as f:
    lines = f.readlines()   # reads the WHOLE file at once into a list of lines
```

Looping directly over the file object (the first example) is generally preferred for larger files, because it reads one line at a time instead of loading the entire file into memory at once — you'll see exactly why this matters when you meet generators on Day 10.

Writing to a file

```
with open("out.txt", "w") as f:    # "w" = write mode
    f.write("hello\n")             # \n is the special character meaning "start a new line"
```

Careful: "w" mode completely erases (truncates) the file first, if it already exists, before writing your new content — there's no confirmation prompt, no undo. If you instead want to add content to the *end* of an existing file without erasing what's already there, use `"a"` (append) mode instead:

```
with open("out.txt", "a") as f:
    f.write("more\n")
```

Summary of the modes you'll use most: `"r"` read (default), `"w"` write/erase-then-write, `"a"` append, and adding `"b"` to any of them (e.g. `"rb"`) switches to binary mode, for non-text files like images, where you get raw **bytes** back instead of text.

Encoding — a subtlety worth knowing about even as a beginner

Text files are actually stored as raw bytes on disk, and there are different systems ("encodings") for converting between bytes and human-readable characters. If you don't specify one, Python picks a default based on your operating system's settings — which can silently differ between your computer and someone else's, or a server's, causing a file that works fine for you to behave oddly for a teammate. The fix is simple: be explicit.

```
with open("data.txt", encoding="utf-8") as f:  
    contents = f.read()
```

"utf-8" is the overwhelmingly standard choice today and correctly handles virtually all text you'll encounter, including non-English characters and emoji — get in the habit of specifying it explicitly whenever you open a text file, starting with today's exercises.

Exercises

Open `exercises.py`, implement each `# TODO`, and run `python exercises.py`. This one creates and deletes a couple of small temporary files while it runs — that's expected and part of testing your file-handling code for real.

Day 6 - Errors, Exceptions, Modules, and Packages

Day 6 — Errors, Exceptions, Modules, and Packages

Objectives

- Understand what an exception actually is, and why Python uses them so heavily instead of just returning error codes
- Handle errors gracefully with `try/except/else/finally`, understanding what each part is actually for
- Create your own custom error types when the built-in ones don't fit
- Understand modules (files of Python code you can reuse) and packages (folders of related modules)

What is an exception, really?

You've already seen several errors crash a program by this point — `IndexError`, `KeyError`, `TypeError`, `ZeroDivisionError`. Each of these is an **exception**: an object Python creates the instant something goes wrong, describing exactly what happened. By default, when an exception is created and nothing "catches" it, Python stops your program immediately and prints that exception's details to the terminal (the traceback you learned to read on Day 0) — no further lines of code run.

The crucial idea for today: you can choose to **catch** an exception yourself, examine or respond to it, and keep your program running instead of letting it crash. This is an enormous amount of what separates "a script that works until something slightly unexpected happens" from "a real program that behaves sensibly even when things go wrong" — which is exactly what's expected of code written for a job.

`try` / `except` — catching an error instead of crashing`

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("You can't divide by zero!")
    result = None

print("Program keeps running normally after this.")
```

Read this as: "try to run the code in the `try` block; if a `ZeroDivisionError` happens anywhere inside it, jump immediately to the matching `except` block instead of crashing, run that instead, and then continue the program normally afterward." Without the `try/except`, that same division would have completely stopped the program right there.

You can catch different exception types differently, and Python checks each `except` in order, top to bottom, using the first one that matches:

```
try:
    value = int(user_input)          # could raise ValueError if user_input isn't a valid number
```

```

    result = 10 / value          # could raise ZeroDivisionError if value is 0
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("Can't divide by zero.")

```

You can also catch several exception types with one shared response, by grouping them in parentheses:

```

except (ValueError, TypeError) as e:
    print(f"Something was wrong with the input: {e}")

```

as `e` captures the actual exception object into a variable named `e`, so you can inspect it — for instance, `print(f"{e}")` usually shows you the human-readable message describing exactly what went wrong.

Why you should (almost) never write a bare `except:`

It's tempting, especially as a beginner frustrated by an error, to write:

```

try:
    do_something_risky()
except:          # DON'T DO THIS
    pass        # `pass` means "do nothing" -- silently ignore literally ANY problem

```

This is dangerous because it catches *every possible thing that could go wrong**, including bugs you never anticipated and never intended to hide — a typo that causes a `NameError`, a completely unrelated crash somewhere deep inside a library you're using — and silently swallows all of it without any explanation, leaving you utterly confused later about why your program isn't behaving as expected, with zero clues as to why.

Always catch the specific exception type(s) you actually expect and know how to sensibly respond to.

`else` and `finally` — the two less-common, but useful, parts`

```

try:
    value = risky_operation()
except ValueError:
    print("that failed")
else:
    print("this runs ONLY if the try block succeeded with no exception at all")
finally:
    print("this ALWAYS runs, whether there was an error or not -- great for cleanup steps")

```

`else` is for code that should run only after a successful `try`, but that you deliberately do *not* want wrapped in the same `try` — this way, if that follow-up code itself has a bug, you won't accidentally and confusingly catch that separate bug with the same `except` meant for a different problem. `finally` always runs no matter what, which makes it useful for guaranteed cleanup — though as you learned yesterday, `with` is usually the better tool specifically for resources like files.

Raising your own exceptions

You're not limited to catching exceptions Python raises automatically — you can deliberately raise your own, any time your own code detects a situation it considers an error, using the `raise` keyword:

```

def withdraw(balance, amount):
    if amount > balance:
        raise ValueError(f"insufficient funds: have {balance}, need {amount}")
    return balance - amount

```

```
withdraw(100, 500) # raises: ValueError: insufficient funds: have 100, need 500
```

This immediately stops the function (and, unless something catches it, the whole program) and reports the message you gave, exactly like any error Python raises on its own.

Custom exception types — your own, specific categories of error

For errors specific to your own program's logic, it's good practice to define your own exception type rather than reusing a generic built-in one like `ValueError` for everything. You do this by writing a small class (you'll learn classes properly on Day 8, but this specific pattern is simple enough to use today):

```
class InsufficientFundsError(Exception):
    pass # `pass` here just means "this class needs nothing extra beyond what Exception already provides"

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"need {amount}, have {balance}")
    return balance - amount
```

Why bother creating your own type instead of just using `ValueError`? Because it lets whoever calls your code catch *precisely* your specific error, without accidentally also catching unrelated `ValueErrors` raised by something completely different elsewhere in the same `try` block:

```
try:
    withdraw(100, 500)
except InsufficientFundsError:
    print("Please add funds to your account.")
```

Modules — every `.py` file is one

You've been writing single files so far. A **module** is simply any `.py` file, viewed as something that can be *imported* — meaning another file can gain access to the functions, variables, and classes defined inside it.

```
import math # gives you access to everything inside the math module
print(math.sqrt(16)) # 4.0 -- accessed via "math.", since we imported the whole module

from math import sqrt # pulls just ONE specific name directly into your file
print(sqrt(16)) # 4.0 -- no "math." prefix needed now

import math as m # "aliasing" -- giving an imported module a shorter nickname
print(m.sqrt(16)) # 4.0
```

`math` here is one of Python's **standard library** modules — code that comes bundled with every Python installation, ready to use, covering everything from mathematics to dates to file paths, without needing to install anything extra.

Avoid `from module import *` (which imports absolutely everything a module offers at once) in real code — it makes it unclear, when reading code later, exactly which module any given name actually came from, and it can silently overwrite names you already have without any warning.

Packages — a folder full of related modules

Once a project grows beyond a handful of files, you organize related modules into a **package** — simply a folder containing multiple `.py` files (and typically a special, often-empty file named `__init__.py`, which is

what tells Python "treat this folder as an importable package"):

```
myproject/  
  mypackage/  
    __init__.py  
    utils.py  
    models.py  
  main.py
```

From inside `main.py`, you could write `from mypackage import utils` or, to reach even deeper, `from mypackage.utils import some_function`. You'll build a small package for real yourself on Day 13.

`if \_\_name\_\_ == "\_\_main\_\_":` — a pattern you'll see in nearly every real Python file

Every module Python runs has a hidden variable called `__name__`. When you run a file **directly** (`python my_file.py`), Python automatically sets `__name__` to the exact text `"__main__"` inside that file. But if that same file is instead **imported** by some other file, `__name__` is set to the module's own name instead (e.g., `"my_file"`) — never `"__main__"`.

This lets a single file provide reusable functions AND optionally act as a runnable script, without the "run as a script" part happening by accident just because someone else imported it for its functions:

```
def main():  
    print("Running as a script!")  
  
if __name__ == "__main__":  
    main()
```

If another file does `import my_file`, that import alone will **not** print `"Running as a script!"` — `main()` only actually runs if `my_file.py` itself is the file you executed directly from the terminal. You'll use this exact pattern in tomorrow's mini project, and in essentially every standalone script you write from here on.

Exercises

Open `exercises.py`, fill in the `# TODOs`, and run `python exercises.py` to check your work.

Day 7 - Mini Project: CLI Task Tracker (Week 1 Review)

Day 7 — Mini Project: CLI Task Tracker (Week 1 Review)

Objectives

Combine everything from Days 1-6 into one working program:

- variables/types/mutability (Day 1)
- control flow, loops (Day 2)
- lists and dicts as your data model (Day 3)
- functions with clear parameters and return values (Day 4)
- string formatting and file I/O (Day 5)
- exception handling and modules (Day 6)

This is intentionally less hand-held than the previous days — the point of a review project is to make the design decisions yourself, using what you've learned, and get stuck in the productive way. That said, this lesson explains every new piece of vocabulary you'll need (command-line arguments, JSON) in full before you start, exactly like every other day — "less hand-held" means fewer step-by-step instructions on *how* to build it, not less explanation of the *new* concepts involved.

What's actually new today, explained fully

Two things show up in this project that you haven't formally used yet: **command-line arguments** and **JSON**. Both are explained below before you start.

Command-line arguments — how a program receives instructions typed after its name

So far, every script you've run has been `python somefile.py`, with nothing after the filename. But you've also been typing things like `python exercises.py` with no extra words — today, you'll write a program that pays attention to *extra words typed after the filename*, called **command-line arguments**. For example: `python tasks.py add "Buy milk"` — here, `add` and `"Buy milk"` are two arguments your program can read and react to differently.

Python gives you access to these through a list called `sys.argv`, provided by the built-in `sys` module (recall from Day 6: a "module" is a file of pre-written Python code you gain access to via `import`):

```
import sys
print(sys.argv)
```

If you ran this as `python tasks.py add "Buy milk"`, `sys.argv` would be the list `["tasks.py", "add", "Buy milk"]`. Notice `sys.argv[0]` is *always* the name of the script itself — the arguments you actually care about start at index 1. So `sys.argv[1]` would be `"add"`, and `sys.argv[2]` would be `"Buy milk"`. This is just an ordinary Python list — everything you learned about lists on Day 3 (indexing, slicing, `len()`) applies to it

directly.

This project is your first time using `sys.argv` for real, which is why the instructions below explicitly walk you through reading commands out of it by hand, rather than reaching for a fancier tool (`argparse`, which you'll meet in later, more advanced work) — doing it manually once, here, builds real intuition for what command-line arguments actually are underneath any tool that makes it more convenient later.

JSON — saving Python data to a file so it survives between runs

Every program you've written so far forgets everything the instant it finishes running — all your variables simply cease to exist. Today's project needs to **persist** data — meaning: save it somewhere so that the *next* time you run the program (even tomorrow, even after restarting your computer), it remembers what was saved before.

JSON (JavaScript Object Notation — the name comes from another programming language, but it's a completely universal, language-independent text format at this point) is a simple, human-readable way of writing down data — lists, dicts, strings, numbers, True/False, None — as plain text, so it can be saved to a file and read back later, by Python or by any other programming language. Python's built-in `json` module converts back and forth between "real" Python data (in memory, while your program is running) and JSON text (saved permanently on disk):

```
import json

data = [{"description": "Buy milk", "done": False}]

with open("tasks.json", "w") as f:
    json.dump(data, f)          # takes a Python object (here, a list of dicts) and WRITES it as JSON

with open("tasks.json") as f:
    loaded = json.load(f)      # READS the JSON text back out of the file and turns it back into Python
```

Notice the naming pattern: `dump` = take a Python value and write it out as JSON (think: "dump this data out to a file"). `load` = read JSON text back in and reconstruct the Python value it represents. You'll get the full, deeper treatment of JSON (including reading it from web APIs) on Day 11 — for today, this much is genuinely everything you need.

The brief

Build a **command-line task tracker** called `tasks.py` that a user runs repeatedly from the terminal to manage a to-do list that persists between runs (saved to a JSON file, see the note below on `json` — you'll cover it properly Day 11, but `json.dump/json.load` are enough to use today).

Required features

- 1 `python tasks.py add "Buy milk"` — adds a new task, not yet complete
- 2 `python tasks.py list` — prints all tasks with an index number and a `[ ]/[x]` complete marker
- 3 `python tasks.py done 2` — marks task #2 as complete
- 4 `python tasks.py remove 2` — deletes task #2
- 5 Tasks persist in a file (e.g. `tasks.json`) between runs — running `list` after closing and reopening your terminal should still show your tasks

Suggested data model

A list of dicts, e.g.:

```
[
    {"description": "Buy milk", "done": False},
    {"description": "Walk dog", "done": True},
]
```

Suggested architecture (you decide the details)

- A function to load tasks from the JSON file (handle the file-not-existing-yet case — first run — without crashing: this is exactly the exception-handling judgment call from Day 6)
- A function to save tasks back to the JSON file
- One function per command (`add_task`, `list_tasks`, `complete_task`, `remove_task`)
- Use `sys.argv` to read command-line arguments (`import sys`; `sys.argv[0]` is the script name, `sys.argv[1]` is the first real argument, etc.) — you haven't been taught `argparse` yet on purpose; parsing `sys.argv` by hand is a good forcing function to practice list indexing, slicing, and `try/except` around `int(sys.argv[2])`

Minimal JSON usage you need (full treatment on Day 11)

```
import json

with open("tasks.json", "w") as f:
    json.dump(data, f)          # write a Python object as JSON text

with open("tasks.json") as f:
    data = json.load(f)         # read JSON text back into a Python object
```

Edge cases worth handling (this is where the real learning is)

- What happens if the user runs `list` before ever running `add`? (file doesn't exist yet)
- What happens if the user runs `done 99` but there are only 3 tasks? (index out of range)
- What happens if the user runs `done abc` (not a number)?
- What happens if the user runs the script with no arguments, or an unrecognized command?

None of these should crash with an ugly traceback — they should print a clear, human-readable message. This is the difference between "code that works on the happy path" and "code you could actually ship."

How to approach this (process, not just code)

- 1 Sketch the file structure and function signatures on paper/in comments FIRST, before writing logic — this is what real engineers do before touching a keyboard for anything non-trivial.
- 2 Build `add` and `list` first, test manually by running the script from the terminal, before adding `done/remove`.
- 3 Get the happy path fully working before handling edge cases.
- 4 Once it all works, go back and deliberately try to break it (the edge cases above) and fix what breaks.

A note on scope

Don't reach for classes, `argparse`, or anything from Week 2 — the whole point is proving you can build something real with *only* Week 1 material. You'll rebuild something similar with classes and better tooling by Day 14.

What's provided

- `starter.py` — an empty scaffold with function signatures and docstrings to get you started (optional to use — you can also start from a blank file)
- `solution.py` — a full reference implementation. **Build your own first.** This project has many equally valid designs; comparing your approach to this one afterward is valuable, but only after you've made your own decisions and hit your own bugs.

Manual test checklist

Run these in order from your terminal in this folder and confirm the output matches expectations:

```
python tasks.py add "Buy milk"
python tasks.py add "Walk dog"
python tasks.py list
python tasks.py done 1
python tasks.py list
python tasks.py remove 2
python tasks.py list
python tasks.py done 99
python tasks.py done abc
python tasks.py bogus-command
python tasks.py
```

Day 8 - OOP Fundamentals: Classes, self, Instances

Day 8 — OOP Fundamentals: Classes, `self`, Instances

Objectives

- Understand, from first principles, what a class actually is and why programmers use them
- Understand `self` well enough to never find it mysterious again
- Understand the difference between a class attribute (shared) and an instance attribute (private to one object), and the bug this distinction prevents
- Build a sense for when reaching for a class is the right call, versus when a plain dict or function is enough

Welcome to Object-Oriented Programming

Today starts **Object-Oriented Programming**, usually abbreviated **OOP** — a way of organizing code around "objects" that bundle together both data and the operations that work on that data, instead of keeping data and functions completely separate (which is roughly what you've been doing all of Week 1). You've actually been *using* objects this entire course without necessarily thinking of them that way — remember from Day 1, everything in Python is an object, including the lists and dicts you already know how to use, each with their own bundled-in methods like `.append()` and `.get()`. Today you learn how to design and build your *own* objects, tailored to whatever you're working on.

A class is a blueprint; an instance is a real thing built from that blueprint

Imagine an architect's blueprint for a house. The blueprint itself isn't a house you can live in — it's a *plan* describing what every house built from it will have (so many bedrooms, a kitchen in this spot, etc.). You can build many actual, physical houses from that one blueprint, and each one is independent — painting one house blue doesn't paint the others blue too.

A **class** is exactly this kind of blueprint, written in code. An **instance** is one actual object built from that blueprint — and you can create as many independent instances as you like from a single class.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

rex = Dog("Rex", "Labrador")      # rex is an INSTANCE of the Dog class
fido = Dog("Fido", "Poodle")      # fido is a completely separate instance

print(rex.bark())                # Rex says Woof!
```

```
print(fido.bark())      # Fido says Woof!
```

`Dog` is the blueprint (the class); `rex` and `fido` are two separate, independent houses built from it (the instances). Both have a `.name`, a `.breed`, and a `.bark()` method available, because that's what the `Dog` blueprint specifies every `Dog` instance must have — but `rex.name` and `fido.name` hold different values, entirely independent of one another, exactly like two houses built from one blueprint can be painted different colors without affecting each other.

`self` — not magic, just an explicit reference to "this particular instance"

If you've glanced ahead at other languages before, you may have seen the word `this` used similarly — Python's `self` does the same job, but, true to Python's general philosophy (favor being explicit over implicit — you'll hear this idea again), it's not hidden or automatic in quite the same way; it's an ordinary parameter that you must write out yourself.

Here's the mechanism, precisely: when you write `rex.bark()`, Python actually does something equivalent to `Dog.bark(rex)` behind the scenes — it automatically passes `rex` in as the **first** argument to `bark`. That's what the parameter named `self` inside the method's definition actually receives: the specific instance the method was called on. This is exactly **why** every single method you write inside a class must list `self` as its very first parameter — it's not optional boilerplate; it's how the method gets access to the particular instance's own data (`self.name`, `self.breed`) at all.

```
rex.bark()      # Python internally treats this as: Dog.bark(rex)
Dog.bark(rex)   # ...which you could, in principle, write directly yourself -- it does the same
```

You could technically name this first parameter anything you like — Python doesn't enforce the name `self` — but every single Python programmer, everywhere, names it `self` by convention, and deviating from that convention would make your code confusing to anyone else who reads it (including future-you). Always call it `self`.

`\_\_init\_\_` — setting up a brand new instance

`__init__` (pronounced "dunder init" — "dunder" is short for "double underscore," since it's surrounded by two underscores on each side; you'll meet more of these "dunder methods" tomorrow) is a special method that Python automatically runs the instant you create a new instance of a class. Its job is to set up that instance's starting data.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)      # __init__ runs automatically here, with self=p (the new instance), x=3, y=4
print(p.x, p.y)     # 3 4
```

`self.x = x` is worth reading very carefully, since the same word `x` appears twice with two different meanings: the `x` on the right is the **parameter** (the value that was passed in when `Point(3, 4)` was called); `self.x` on the left is creating a brand-new piece of data that belongs specifically to **this instance**, named `x`, and setting it equal to that parameter's value. Any attribute you want each individual instance to have and remember

(accessible later as `some_instance.attribute_name`) needs to be set on `self` somewhere, almost always inside `__init__`.

Class attributes vs. instance attributes — a distinction that prevents a real, common bug

A **class attribute** is defined directly inside the class body, but *outside* of any method, and is shared by absolutely every instance of that class:

```
class Dog:
    species = "Canis familiaris"    # a CLASS attribute -- one single value, shared by ALL Dog instances

    def __init__(self, name):
        self.name = name            # an INSTANCE attribute -- each Dog gets its OWN separate value
```

Every `Dog` instance reads the exact same `species` value (unless you deliberately override it on one specific instance), because there's only one `species` value total, living on the class itself, not on each instance individually. This is fine, even useful, for genuinely shared constants that will never differ between instances. But it becomes a serious, hard-to-spot bug the moment the class attribute is a *mutable* type (a list or dict) — and this is exactly the same underlying trap as Day 1's mutable-default-argument problem, now showing up in class form:

```
class ShoppingCart:
    items = []                        # DANGER -- this ONE list is shared across EVERY ShoppingCart instance!

    def add(self, item):
        self.items.append(item)

cart1 = ShoppingCart()
cart2 = ShoppingCart()
cart1.add("apple")
print(cart2.items)    # ['apple']  !! -- cart2 never had "apple" added to it directly, yet there it is!
```

This happens because `items` was defined once, on the *class*, and both `cart1` and `cart2` share that exact same underlying list object — `cart1.add("apple")` mutates that one shared list in place, and `cart2` (which was never given its own separate list) sees the exact same shared object change. The fix mirrors Day 1's exactly: put mutable state inside `__init__`, so each instance gets its own, brand-new, independent object:

```
class ShoppingCart:
    def __init__(self):
        self.items = []            # a fresh, new, empty list -- created separately for EACH instance
```

Methods — functions that live inside a class and can read/change that instance's data

A **method** is simply a function defined inside a class. Because it always receives `self` (the specific instance it was called on) as its first parameter, a method can read and modify that particular instance's own data:

```
class Counter:
    def __init__(self):
        self.count = 0
```

```

def increment(self):
    self.count += 1    # modifies THIS instance's own count -- other Counter instances are una

def reset(self):
    self.count = 0

c1 = Counter()
c2 = Counter()
c1.increment()
c1.increment()
print(c1.count)    # 2
print(c2.count)    # 0 -- completely separate from c1, since each instance's __init__ gave it its own

```

Compare this with Day 4's closure-based counter (`make_counter`, which used `nonlocal`) — both achieve a similar goal (something that remembers a running value across calls), but a class scales far better once you have several **related** pieces of state and **several** different operations that need to work on that same state together — which is exactly what starts happening as your programs grow past today.

When should you actually reach for a class, versus a plain dict or a plain function?

This is a genuine judgment call you'll get better at with practice, but here's a starting rule of thumb:

- **Use a plain function** when you're just computing something from some inputs, with no ongoing state to remember between calls.
- **Use a plain dict** when you have a simple bundle of related data, but no real behavior (methods) attached to it, and you're not trying to enforce any particular guarantee about exactly which fields it will always have.
- **Use a class** once you have data **and** behavior that clearly belong together, and you want a guarantee about shape — every instance of `Dog` is guaranteed to have a `.name` and a `.bark()` method, in a way a loose dict never guarantees anything about its own keys. A strong, practical signal that you've outgrown a dict and should switch to a class: you notice you have several separate functions that all take the same dict shape as their first argument, and each one reaches in and reads or mutates specific keys of it — that's data and behavior that clearly belong bundled together, which is precisely what a class is for.

Exercises

Open `exercises.py`, fill in each `# TODO` inside the class definitions provided, and run `python exercises.py` to check your work against the PASS/FAIL output.

Day 9 - OOP Advanced: Inheritance, Dunder Methods, Composition

Day 9 — OOP Advanced: Inheritance, Dunder Methods, Composition

Objectives

- Understand inheritance: how one class can share and extend another's behavior
- Understand `super()` and why it exists
- Understand polymorphism — the same line of code behaving differently depending on what kind of object it's dealing with
- Meet the most important "dunder" (double-underscore) methods and what each hooks into
- Understand composition as an alternative to inheritance, and how to choose between them

Inheritance — one class building on another

Inheritance lets you define a new class that automatically gets all the attributes and methods of an existing class, and then adds or changes only what's actually different, instead of rewriting everything from scratch.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("subclasses must implement speak()")

    def introduce(self):
        return f"I am {self.name} and I say {self.speak()}"

class Dog(Animal):          # Dog INHERITS from Animal -- the parentheses show the parent class
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

rex = Dog("Rex")
print(rex.introduce())    # I am Rex and I say Woof
```

`Animal` here is called the **parent class** (or "base class," or "superclass"); `Dog` and `Cat` are **child classes** (or "subclasses"). Writing `class Dog(Animal):` means "`Dog` is a `Dog`, but it's *also* an `Animal`, and starts out with everything `Animal` already has." Notice `Dog` never wrote its own `__init__` or `introduce` method at all — it inherited both directly from `Animal`, completely free, and only needed to define what's genuinely different

about a `Dog`: its own version of `speak`.

`Animal.speak` deliberately raises `NotImplementedError` — this is a common, deliberate pattern meaning "every subclass of `Animal` MUST provide its own `speak` method; the base `Animal` class refuses to guess a default for you, since a generic `Animal` doesn't have a single specific sound." If you forgot to define `speak` in some new subclass, you'd find out immediately and clearly (a crash with a clear message), rather than silently getting some wrong, made-up default sound.

Polymorphism — the same call, different behavior, depending on the actual object

Notice something important in `introduce`: it calls `self.speak()` without knowing or caring in advance whether `self` is a `Dog`, a `Cat`, or some other `Animal` subclass someone writes in the future. Each type of object supplies its own correct answer when asked. **This is called polymorphism** ("many forms") — one piece of code (`self.speak()`, called from inside `introduce`) automatically does the *correct, different* thing depending on what kind of object it's actually attached to at the time, decided while the program is running, not decided in advance by the code itself.

This is genuinely powerful: you could write a hundred more `Animal` subclasses (`Bird`, `Cow`, `Snake`, anything), and `introduce` would keep working correctly for every single one, without you ever needing to go back and modify `introduce` itself — as long as each new subclass properly supplies its own `speak` method.

`super()` — reusing the parent's setup instead of duplicating it

If a subclass needs its *own* `__init__` (to accept and store some extra piece of data the parent doesn't have), but still wants the parent's original setup logic to run too, you shouldn't copy-paste that logic — you should call it directly via `super()`:

```
class Employee(Animal):    # a deliberately silly example, purely to show the mechanism clearly
    def __init__(self, name, salary):
        super().__init__(name)    # this literally runs Animal.__init__(self, name) for you
        self.salary = salary

e = Employee("Ana", 50000)
print(e.name, e.salary)    # Ana 50000
```

Without that `super().__init__(name)` line, `self.name` would never actually get set at all inside `Employee` — you'd have to manually retype `self.name = name` yourself, duplicating logic that already exists in `Animal`. And if `Animal.__init__` ever changed later (say, it started validating that `name` isn't empty), any subclass that had copy-pasted the old logic instead of calling `super()` would silently miss that update. Calling `super()` keeps the parent's setup as the single, authoritative source of truth, referenced rather than duplicated.

`isinstance()` versus `type()` — and why `isinstance` is almost always the right choice

```
isinstance(rex, Dog)      # True -- rex genuinely is a Dog
isinstance(rex, Animal)   # True -- rex is ALSO an Animal, because Dog inherits from Animal
```

```
type(rex) == Animal          # False -- rex's EXACT type is Dog, not Animal, even though it IS an Animal
```

`isinstance` asks "is this object a Dog, OR anything that inherits from Dog?" — a broader, inheritance-aware question. `type(x) == SomeClass` asks a much narrower, stricter question: "is this object's exact type precisely `SomeClass`, no more, no less?" Prefer `isinstance` in almost every situation, because it keeps working correctly even as your codebase grows — code that checks `isinstance(x, Animal)` will happily keep working if you add a brand-new `Bird(Animal)` subclass next month; code that checks `type(x) == Dog` specifically will silently, incorrectly exclude that new `Bird`, even though a `Bird` genuinely is an `Animal` too.

Dunder methods — how your own classes hook into Python's built-in syntax

You've already used one dunder method: `__init__`. "Dunder" methods (from "double underscore," since they're written as `__name__`) are Python's mechanism for letting your own custom classes work naturally with built-in operators and syntax (`+`, `==`, `len(...)`, `print(...)`, `for...in`, and more), instead of forcing you to invent a differently-named method for every single class you write.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"    # unambiguous text representation, mainly for developers

    def __str__(self):
        return f"({self.x}, {self.y})"         # friendlier text representation, for regular users

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)          # (4, 6) -- Python automatically calls v1.__add__(v2) when it sees the +
print(v1 == Vector(1, 2)) # True -- Python calls v1.__eq__(...) when it sees ==
```

`__repr__` versus `__str__`: if a class defines only one of them, that one is used everywhere. If a class defines both, `print(instance)` and `str(instance)` use `__str__` (meant to be nicely readable for an end user), while typing the instance's name directly into the interactive REPL, or calling `repr(instance)` explicitly, uses `__repr__` (meant to be precise and useful for a *developer* debugging the program — ideally something close to what you'd type back into Python to recreate an identical object). As a habit worth starting now: **always write a `__repr__` for classes you'll be debugging**, since without one, printing an instance just shows an unhelpful default like `<__main__.Vector object at 0x000001A2B3C4D5E6>`, which tells you almost nothing useful.

A few other dunder methods worth being able to recognize when you see them in other people's code, even before you've used them yourself: `__len__` (lets `len(your_object)` work), `__getitem__` (lets `your_object[index]` work), `__iter__` (lets `for item in your_object:` work), `__contains__` (lets `item in your_object` work), and `__enter__`/`__exit__` (the exact mechanism behind `with`, from Day 5 — now

you know what makes that work under the hood).

Composition — an alternative to inheritance for "has-a" relationships

Inheritance models an "is-a" relationship: a **Dog** is an **Animal**. **Composition** instead models a "has-a" relationship: one class simply holds an *instance* of another class as one of its own attributes, rather than inheriting from it at all:

```
class Engine:
    def start(self):
        return "Vroom"

class Car:
    def __init__(self):
        self.engine = Engine()      # Car HAS-A Engine -- it holds one as an attribute, it doesn't inherit

    def start(self):
        return self.engine.start()  # Car "delegates" the actual work to its Engine

my_car = Car()
print(my_car.start())  # Vroom
```

Why would you choose composition over inheritance, when either would technically work? Inheritance creates a fairly rigid, tightly-coupled relationship — a subclass depends on its entire parent's internal design, and if you ever needed a **Car** that used a totally different kind of engine (an **ElectricEngine**, say), inheritance would force you into an awkward class hierarchy to express that. Composition is far more flexible: swapping **Engine()** for **ElectricEngine()** inside **Car.__init__** requires no change at all to the **Car** class's overall shape, and doesn't risk breaking anything else that happens to inherit from **Car**.

A widely-repeated piece of professional advice, worth remembering as you start designing your own classes: **"favor composition over inheritance."** A reasonable starting rule of thumb: reach for inheritance when there's a genuine "is-a" relationship *and* you specifically want polymorphism (treating several different subclasses interchangeably through one shared interface, exactly like **Animal/Dog/Cat** above). Reach for composition when you're simply reusing a separate piece of functionality without needing that interchangeability.

Exercises

Open **exercises.py**, fill in each **# TODO**, and run **python exercises.py**.

Day 10 - Iterators, Generators, and Decorators

Day 10 — Iterators, Generators, and Decorators

Objectives

- Understand the actual mechanism a `for` loop uses internally — something you've been relying on since Day 2 without seeing how it works
- Write your own generator functions using `yield`, and understand exactly why they save memory
- Understand decorators as a direct, practical application of Day 4's closures
- Write your own decorators, including ones that accept their own arguments

Peeling back the curtain on `for` loops

Since Day 2, you've written many `for` loops without ever needing to know *how* Python actually pulls items out of a list, a string, or a dict one at a time. Today you'll learn the real mechanism — partly because it's genuinely useful to understand, and partly because it's the exact foundation that today's main topic, generators, is built directly on top of.

Two new vocabulary words: something is called **iterable** if it has a built-in method named `__iter__` that can produce an **iterator** — a separate, different kind of object whose entire job is to hand back exactly one value at a time, on demand, via its own `__next__` method, and to signal "there's nothing left" by raising a special exception called `StopIteration` once it's exhausted. A `for` loop is really just convenient shorthand for a pattern you could, in principle, write yourself:

```
my_list = [10, 20, 30]

iterator = iter(my_list)      # calls my_list.__iter__() behind the scenes, returning an iterator object
while True:
    try:
        item = next(iterator) # calls iterator.__next__(), asking for "the next value"
    except StopIteration:
        break                 # no more items -- exit the loop
    print(item)                # this is the loop "body" you'd normally write directly under `for`
```

This is exactly, precisely what `for item in my_list:` does for you automatically, hiding all of this machinery so you can just write the simple version. The reason this matters today: this exact same protocol — something with a `__next__` that can be asked, one at a time, "what's next?" — is precisely what a generator (below) provides, which is why generators plug directly into `for` loops with zero extra effort.

Generators — functions that hand back values one at a time, pausing in between

A **generator function** looks almost like an ordinary function, except somewhere in its body it uses the keyword `yield` instead of, or alongside, `return`. Calling a generator function does **not** immediately run its body top to bottom the way calling a normal function does — instead, it immediately hands you back a special

generator object, and the function's actual code only starts running, one small step at a time, as something asks it for its next value (which, again, a `for` loop does for you automatically).

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i      # hand back the CURRENT value of i, and PAUSE right here
        i += 1       # this line only runs the NEXT time something asks for another value

for number in count_up_to(5):
    print(number)    # prints 1, 2, 3, 4, 5 -- one at a time, as the loop asks for each one
```

Here's the part that takes a moment to fully click: each time execution reaches `yield i`, the function's entire state — every local variable, exactly where execution was — is frozen in place, and that current value of `i` is handed back to whoever asked. The **next** time something asks this generator for another value, execution resumes exactly where it left off (right after that same `yield` line), not from the very top of the function again. This is fundamentally different from a normal function, which always starts fresh from its very first line, every single time you call it.

Why bother? The concrete, practical reason: memory

Consider the difference between these two ways of getting the same ten million numbers:

```
numbers_list = list(range(10_000_000))    # builds and stores ALL 10 million numbers in memory RIGHT NOW

def numbers_generator(n):
    i = 0
    while i < n:
        yield i
        i += 1

numbers_gen = numbers_generator(10_000_000) # this line runs INSTANTLY and uses almost no memory at all
```

`numbers_list` genuinely allocates space for ten million integers, all at once, whether or not you actually end up using all of them. `numbers_gen`, by contrast, holds essentially nothing in memory beyond "where I currently am" — it produces the next number only the instant something actually asks for it, and immediately forgets it again once it's been handed over. For ten million small integers this difference might not matter much on a modern computer, but the exact same idea applies just as well to something like reading through a multi-gigabyte log file line by line — a generator lets you process it a piece at a time without ever needing to fit the whole file in memory simultaneously.

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()    # hands back ONE line at a time, never holding the whole file in memory
```

An important limitation to know about: once a generator has been fully consumed (you've asked it for every value it has, or you stopped partway through and abandoned it), it's finished for good — you cannot rewind it or restart it. If you need to go through the same sequence of values more than once, you either call the generator *function* again to get a brand-new, fresh generator object, or, if the underlying data is genuinely small enough, just use an ordinary list instead.

Generator expressions — the generator version of a list comprehension

You met list comprehensions on Day 2. Swap the square brackets for parentheses, and you get the lazy, generator equivalent:

```
squares_list = [n * n for n in range(1_000_000)]    # computes and stores ALL one million squares
squares_gen = (n * n for n in range(1_000_000))    # computes NOTHING yet -- purely a plan, ready to go

total = sum(squares_gen)    # NOW, as sum() asks for each value in turn, the actual computation happens
```

Decorators — functions that wrap other functions with extra behavior

A **decorator** is a function that takes another function as input, and returns a brand-new function that adds some extra behavior *around* the original — all without needing to touch or rewrite the original function's own source code. This is built from ideas you already have every piece of: closures (Day 4) and **args/**kwargs* for forwarding arguments along unchanged (also Day 4).

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)    # actually calls the ORIGINAL function, with whatever is passed in
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f} seconds")
        return result
    return wrapper

@timer
def slow_add(a, b):
    time.sleep(0.1)
    return a + b

print(slow_add(2, 3))    # prints something like "slow_add took 0.10xx seconds", then 5
```

That *@timer* line, written directly above *def slow_add(...)*, is nothing more than convenient shorthand for writing *slow_add = timer(slow_add)* right after the function is defined — Python takes the freshly-defined *slow_add*, immediately hands it into *timer*, and replaces the name *slow_add* with whatever *timer* returns (here, *wrapper*). From that point on, whenever anyone calls *slow_add(...)*, they're actually calling *wrapper*, which does its own extra work (timing) and then, inside itself, calls the **original** *slow_add* function (kept alive as *func*, thanks to the closure).

You've very likely already **seen** decorators without knowing what they were — *@staticmethod*, *@classmethod*, and *@property* (built-in ones used with classes) are decorators, and if you go on to use popular tools like Flask (*@app.route(...)*) or pytest (*@pytest.fixture*, which you'll meet on Day 12), you'll see this exact *@something* pattern constantly. Understanding the mechanism today means none of those will ever feel like unexplainable magic.

`functools.wraps` — a small but important detail

Without extra care, a decorated function quietly loses its original name and its docstring, because the name *slow_add* now actually points at *wrapper*, not at the original function:

```
print(slow_add.__name__)    # without care, this would print "wrapper" -- confusing in logs, debugger, etc.
```

The fix is to add one more decorator, from Python's built-in `functools` module, directly onto your own `wrapper` function — its whole job is to copy the original function's name, docstring, and other identifying details onto the wrapper:

```
from functools import wraps

def timer(func):
    @wraps(func)          # copies func's __name__, __doc__, etc onto wrapper
    def wrapper(*args, **kwargs):
        ...
    return wrapper
```

Make `@wraps(func)` a habit every time you write a decorator of your own, starting with today's exercises.

Decorators that take their own arguments

Sometimes you want to configure a decorator itself, like `@repeat(times=3)`. This requires one extra layer of nesting: a plain function that, when called, *returns* a decorator (rather than being a decorator directly):

```
def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def greet(name):
    print(f"Hello, {name}!")

greet("Ana")    # prints "Hello, Ana!" three separate times
```

Read this from the inside out to make sense of it: `repeat(times=3)` runs *first*, immediately, and its job is only to produce and hand back `decorator` (a plain function, now aware of `times=3` thanks to the same closure mechanism from Day 4). *Then*, that returned `decorator` gets applied to `greet`, exactly like any ordinary decorator from earlier in this lesson.

Exercises

Open `exercises.py`, fill in each `# TODO`, and run `python exercises.py`.

Day 11 - Working With Real Data: JSON, CSV, HTTP APIs, Virtual Environments

Day 11 — Working With Real Data: JSON, CSV, HTTP APIs, Virtual Environments

Objectives

- Get the full, proper treatment of JSON (you used a small slice of it already, back on Day 7)
- Read and write CSV (spreadsheet-style) files correctly, using Python's `csv` module rather than parsing them by hand
- Make real HTTP requests to fetch data from the internet, and handle the many ways that can fail
- Understand what a virtual environment is and why practically every real Python project uses one

JSON, properly this time

Back on Day 7, you used just enough JSON to save and load your task tracker's data. Today you get the complete picture. **JSON** (JavaScript Object Notation) is a plain-text format for representing data — lists, dicts (called "objects" in JSON's own terminology), strings, numbers, `true/false`, and `null` — that's understood by virtually every programming language, which is exactly why it's the standard way computer programs (including web servers, called **APIs** — more on those below) send structured data to each other over the internet.

Python's built-in `json` module converts between JSON text and Python's own equivalent data types:

JSON	Python
object { }	<code>dict</code>
array []	<code>list</code>
string	<code>str</code>
number	<code>int</code> or <code>float</code>
<code>true / false</code>	<code>True / False</code>
<code>null</code>	<code>None</code>

```
import json
```

```
data = {"name": "Ana", "tags": ["admin", "active"], "active": True}
```

```
json_text = json.dumps(data, indent=2)    # convert a Python object INTO a JSON string (note the "s")
python_object = json.loads(json_text)     # convert a JSON string BACK INTO a Python object (also
```

```

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)                # write directly to a FILE (no "s" -- operates on a fi

with open("data.json") as f:
    loaded = json.load(f)                       # read directly FROM a file (also no "s")

```

Notice the naming pattern precisely: `dump(s)` always means "Python → JSON" (writing out), and `load(s)` always means "JSON → Python" (reading in). The versions **with** an `s` (`dumps/loads`) work with plain strings sitting in memory; the versions **without** an `s` (`dump/load`) work directly with an already-open file object. `indent=2` simply makes the output nicely readable by a human, with line breaks and 2-space indentation, instead of one long, cramped line — it has no effect on what the data actually means, only on how it looks.

A genuine trap worth knowing about: JSON has no concept of a `tuple` or a `set` — only lists exist in JSON's world. `json.dumps` will silently convert any tuple you give it into a JSON array on the way out, but there's no way to tell, on the way back in, that it was ever supposed to be a tuple rather than a list — `json.loads` will always hand you back an ordinary `list`. If you specifically need a tuple or a set to survive a round trip through JSON unchanged, you have to convert it yourself, deliberately, both before saving and after loading.

CSV — for spreadsheet-style, row-and-column data

CSV stands for "Comma-Separated Values" — a simple text format for tabular data (rows and columns, like a basic spreadsheet), where each line is one row, and commas separate the individual column values within that row. You might be tempted to just use `.split(",")` from Day 5 to parse a CSV file yourself by hand — **don't**. That approach silently breaks the moment any single field itself happens to contain a comma (a very ordinary thing to happen, e.g. a name field containing "`Smith, John`"), because the file format has specific rules (quoting fields that contain commas) that a plain `.split(",")` doesn't know about at all. Python's built-in `csv` module already handles all of these rules correctly for you:

```

import csv

with open("people.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)                # reads just the FIRST row (commonly the column names) and advan
    for row in reader:
        print(row)                       # each remaining row comes back as a plain list of strings

with open("people.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)            # automatically uses the very first row as column names/keys
    for row in reader:
        print(row["name"], row["age"])    # each row now comes back as a DICT, keyed by column name

with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "age"])      # write ONE row (here, acting as the header row)
    writer.writerows([["Ana", 30], ["Bo", 25]]) # write SEVERAL rows at once, from a list of lists

```

Two details specific to CSV that are easy to forget, but worth internalizing now: first, `newline=""` when opening a file for CSV work — this is required on some systems to stop Python's own automatic newline handling from interfering with the CSV module's own separate line-ending rules (a well-known, easy-to-forget quirk, so just always include it for CSV files). Second, and more conceptually important: **every single value read from a CSV file comes back as plain text (a string), even values that look exactly like numbers —**

the CSV format itself has no concept of "this column contains numbers." If you need an actual number to do arithmetic with, you must explicitly convert it yourself, exactly like you practiced with `parse_csv_line` back on Day 5 (`int(row["age"])`, for example).

HTTP requests — how your program talks to other computers over the internet

An **API** (Application Programming Interface) here refers to a way one program can ask another program — often running on a completely different computer somewhere else in the world — for data, or ask it to do something, over the internet. Communicating this way almost always uses **HTTP**, the same underlying protocol (a fancy word for "agreed-upon set of rules for communicating") your web browser uses every time it loads a web page. Python's standard library has a way to do this (`urllib`), but nearly the entire Python community instead uses a much friendlier third-party library called `requests`, which you install with:

```
pip install requests
```

(`pip` is Python's built-in tool for installing extra packages that don't come bundled with Python itself — you'll use it constantly from here on. "Third-party" just means "written and published by someone other than the core Python team," as opposed to `json` or `csv`, which are part of Python's own "standard library" and need no installation at all.)

```
import requests

response = requests.get("https://api.example.com/users/1", timeout=5)

response.status_code    # a number describing what happened: 200 (success), 404 (not found), 500 (s
response.json()         # parses the response body as JSON, handing you back a Python dict/list d
response.text           # the raw response body, as plain text, with no parsing applied
response.ok             # True if status_code is less than 400 (a rough, convenient "did it ba

response.raise_for_status() # if the status code indicates failure (4xx or 5xx), this line RAISES
```

Always include a `timeout`. Without one, if the other computer never responds at all (a common real-world failure — a server that's down, a bad network connection), your program will simply sit there frozen, waiting forever, with no way out. This is a genuine, well-known category of real production incident ("why did our program just hang?"), not a theoretical, only-in-textbooks concern.

Real code always assumes a network call *might* fail, and handles that possibility explicitly, using exactly the `try/except` pattern from Day 6:

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
except requests.RequestException as e:
    print(f"Request failed: {e}")
```

`requests.RequestException` is a broad category covering everything the `requests` library itself might raise — connection failures, timeouts, bad status codes after `raise_for_status()`. Catching it is a direct, practical application of the "EAFP" idea from Day 6 (attempt the operation, and be ready to handle it failing), applied specifically to network calls — which, unlike most code you've written so far, genuinely can fail at any moment for reasons entirely outside your own program's control.

Sending data *to* a server (rather than just requesting it) typically uses a POST request:

```
response = requests.post(
    "https://api.example.com/users",
    json={"name": "Ana", "age": 30},    # requests automatically converts this dict to JSON text and
    timeout=5,
)
```

Virtual environments — giving every project its own, separate set of installed packages

If you simply `pip install` packages without any further setup, they get installed **globally** — meaning every single Python project on your entire computer shares one common pool of installed packages and their exact versions. This becomes a real problem the moment you have two different projects that each need a *different* version of the same package — you genuinely cannot have two different versions of one package installed globally at the same time.

A **virtual environment** solves this by giving one specific project folder its own private, isolated, separate copy of installed packages, completely independent of anything installed globally or in any other project's own virtual environment.

```
python -m venv .venv    # create a new virtual environment, stored in a folder named .venv (d
```

Before you can actually use it, you must **activate** it — every single time you open a fresh terminal window and want to work on that project:

```
.venv\Scripts\activate    # Windows
source .venv/bin/activate  # macOS/Linux
```

Once activated, your terminal prompt usually changes to show the environment's name, and from that point on, anything you `pip install` goes into this project's own private `.venv` folder, not anywhere global:

```
pip install requests    # installs INTO .venv specifically, not system-wide
pip freeze > requirements.txt    # writes down the EXACT versions of everything currently installed
```

`requirements.txt` becomes a portable, shareable "shopping list" — on a different computer entirely (a teammate's laptop, or a server), running `pip install -r requirements.txt` (after creating and activating a fresh `.venv` there too) recreates that exact same set of packages, at those exact same versions, from scratch. This is *why* nearly every real Python project you'll ever open at a job includes a `requirements.txt` (or, in more modern projects, a `pyproject.toml` — you'll meet this on Day 13) and expects you to set up your own virtual environment before running anything at all. When you're done working, `deactivate` leaves the virtual environment and returns your terminal to its normal, global state.

Exercises

Exercises 1-4 (JSON and CSV) are auto-checked and don't need any internet connection — open `exercises.py`, fill in the `# TODOs`, and run `python exercises.py`. Exercise 5, the real HTTP API call, is a separate manual exercise described in `api_exercise.md` in this same folder, since it needs an internet connection and the `requests` package installed — read that file and follow its instructions once you've finished `exercises.py`.

Day 12 - Testing and Debugging: pytest, Type Hints

Day 12 — Testing and Debugging: pytest, Type Hints

Objectives

- Understand, in concrete terms, why automated tests exist and what problem they actually solve
- Write real tests using `pytest`: plain assertions, checking that an error is correctly raised, fixtures, and parametrized tests
- Understand what type hints are, what they genuinely give you, and what they deliberately do *not* do
- Learn to use a real debugger instead of relying only on scattering `print()` statements everywhere

What you've been doing all along, and why it isn't quite enough

Every single exercise file this whole course has ended with a block of code that prints `PASS` or `FAIL` for each function you wrote. That block was, quite literally, a tiny, hand-built, extremely minimal *testing framework* — you've been practicing the core idea of automated testing since Day 1, without a name attached to it yet. Today gives that idea its proper name and its real, professional tooling.

Why do automated tests matter so much, especially at a job? Right now, your programs are small enough that you can just run them and glance at the output to see if something's obviously wrong. But imagine a program with thousands of lines, worked on by a whole team, where changing one small function somewhere might accidentally break something completely different, in some other far-away part of the program, that you didn't even think to check by hand. Automated tests are what catch that kind of accidental breakage **immediately** — the instant you make a change, before you ship it — rather than someone discovering it days or weeks later, in production, potentially affecting real users. The core mindset shift for today: **tests aren't just there to prove your code works once, right now — they exist so that any future change to the code gets checked against everything it might have broken, automatically, forever.**

Setting up pytest

```
pip install pytest
```

`pytest` is a separate, third-party tool (recall from Day 11: not part of Python's built-in standard library, so it must be installed with `pip`) that automatically finds and runs your tests. It looks for files named `test_*.py` or `*_test.py`, and inside those files, for individual functions whose names start with `test_`. You don't need any special import to write a basic test — just Python's own built-in `assert` keyword:

```
# test_math_utils.py
def add(a, b):
    return a + b

def test_add_positive_numbers():
```

```

assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2

```

`assert some_condition` is a statement meaning "if `some_condition` is false, immediately raise an error (specifically, an `AssertionError`) right here; if it's true, do absolutely nothing and continue to the next line." A test function "passes" simply by running to the end without any `assert` inside it ever failing.

Run your tests from the terminal with:

```

pytest                                # automatically finds and runs every test_*.py file in the current folder
pytest test_math_utils.py -v          # run just one specific file, with -v ("verbose") showing each individual test
pytest -k "negative"                  # only run tests whose function name contains the word "negative"

```

When an `assert` fails, `pytest` automatically shows you a detailed, helpful breakdown of exactly which values didn't match and why — you don't need to write your own custom failure message the way some older testing tools require.

Testing that an error is correctly raised

Sometimes the entire *point* of a test is to confirm that your code correctly raises an error under bad input — for that, `pytest` provides `pytest.raises`:

```

import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("cannot divide by zero")
    return a / b

def test_divide_by_zero_raises():
    with pytest.raises(ValueError):
        divide(10, 0)

```

Read this as: "run the code inside this `with` block, and specifically expect a `ValueError` to be raised somewhere in there — if it is, this test passes; if the code runs to the end *without* raising that error, the test fails, because the whole point was confirming that the error genuinely happens."

Fixtures — reusable setup code, matched to your tests by name

A **fixture** is a small, separate function that prepares some piece of data or setup you want available to one or more of your tests, without repeating that setup code inside every single test function:

```

import pytest

@pytest.fixture
def sample_account():
    return {"owner": "Ana", "balance": 100}

def test_deposit(sample_account):
    sample_account["balance"] += 50
    assert sample_account["balance"] == 150

```

The mechanism here, precisely: `pytest` notices that `test_deposit` has a parameter named `sample_account`, sees that there's a fixture function *also* named `sample_account`, and automatically calls that fixture and hands you its return value as the argument — this isn't hidden magic, it's just `pytest` matching parameter names to fixture names for you.

Parametrize — running one test body against many different inputs

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

This one function definition actually runs as **three separate, individually-reported tests** — one for each tuple of values in that list — which is far better than manually writing three separate `assert` lines inside one test, because if, say, only the second combination of inputs fails, `pytest`'s output tells you exactly which one failed, rather than you having to guess which `assert` line among several was the culprit.

What actually makes a test a **good** test

- **Test one specific behavior per test function** — a test named `test_withdraw_insufficient_funds` should check exactly that one scenario, not sneak in an unrelated check about deposits at the same time.
- ****Test what a function **does**, not the specific way it happens to be written internally**** — check its return value or its externally-visible effect, not private implementation details that might reasonably change later without the actual behavior changing.
- **Cover the edge cases, not just the "happy path"** — this entire course's exercises have been quietly training this habit already: empty lists, missing dictionary keys, zero, negative numbers. Keep doing this deliberately now that you know it has a name.
- **Tests should give the same result every single time they run** — never depend on the exact current time, on genuinely random values (unless you deliberately fix, or "seed," the randomness), on an actual network connection, or on tests happening to run in one particular order.

Type hints — documentation that a tool can actually double-check for you

```
def add(a: int, b: int) -> int:
    return a + b

def greet(name: str, times: int = 1) -> list[str]:
    return [f"Hello, {name}!" for _ in range(times)]
```

The `: int` after a parameter name, and the `-> int` after the closing parenthesis, are called **type hints** (or "type annotations") — they state what type each parameter is expected to be, and what type the function is expected to return.

Crucially, type hints do absolutely nothing at all while your program is actually running — recall from Day 1 that Python is dynamically typed, and that hasn't changed; Python will happily run `add("a", "b")` without a single complaint, type hints notwithstanding, and simply return `"ab"` (since `+` on two strings means "join them together"). So what are type hints actually *for*, if Python itself ignores them? Three real, practical things: first, they're documentation that lives directly in the code itself, right where you're reading it, instead of drifting out of sync in some separate document nobody remembers to update. Second, your code editor can read them and warn you, as you type, if you're about to pass the wrong kind of value somewhere. Third — and this is the big one, professionally — a completely separate tool called a **type checker** (`mypy` and `pyright` are the two most common ones) can scan your entire codebase *before you ever run it* and catch a whole category of mistakes ahead of time, which is exactly how the Python ecosystem compensates for not having a true compiler that would otherwise catch these errors automatically, the way some other languages do.

A shape you'll see constantly, worth knowing today:

```
from typing import Optional    # or, in modern Python (3.10+), just write `X | None` directly, no imp

def find_user(user_id: int) -> Optional[dict]:    # equivalently: -> dict | None
    ...
```

`Optional[X]` (or `X | None`) means "this returns a value of type `X`, OR it might return `None` instead" — this is the exact same "might not find anything" situation from Day 3's `dict.get()`, now stated explicitly and visibly right in the function's signature, so that anyone calling this function is warned, right there, that they need to handle the possibility of getting `None` back.

At most real jobs using Python today, you'll be expected to add type hints to new code you write — it's considered a standard, expected practice in most modern, professional Python codebases now, not an optional extra.

Debugging with a real debugger, instead of only `print()`

Scattering `print(some_variable)` statements throughout your code to see what's happening works, and you've likely already done exactly this while working through earlier days — but it has a real limitation: you have to *guess in advance* what you'll need to see, add print statements for exactly that, run the program, and if you guessed wrong or need to see something else too, edit and re-run all over again. Python's built-in debugger lets you pause your program mid-execution and look at *anything*, interactively, without needing to have predicted it beforehand:

```
def buggy_function(items):
    breakpoint()          # execution PAUSES here and drops you into an interactive prompt
    total = sum(items)
    return total / len(items)
```

When execution hits `breakpoint()`, you'll get an interactive (`Pdb`) prompt right there in your terminal. A few commands worth knowing today: `n` (run the next line), `s` (step *into* a function call, rather than over it), `c` (continue running normally until the next breakpoint, or the end), `p some_variable` (print the current value of any variable you're curious about), `l` (show the surrounding lines of code for context), `q` (quit the debugger entirely). This gives you the power to inspect *any* variable, at that exact moment, and even test out a

potential fix interactively before committing to writing it into your actual code.

Most code editors (VS Code, PyCharm) also provide a visual debugger — you click directly in the margin next to a line of code to set a breakpoint there, then run your program in a special "debug mode," and get this exact same inspection power through a graphical interface instead of a text prompt. Building the habit of reaching for a debugger, instead of defaulting to `print()`, the moment something confusing happens, is one of the single highest-value habits you can build starting this week.

Exercises

Open `calculator.py` and add type hints to every function first (see the instructions at the top of `test_calculator.py` for exactly what's expected). Then open `test_calculator.py` and fill in each `# TODO` test. Run everything with `pytest test_calculator.py -v` from this folder and read the output.

Day 13 - Practical Python: Project Structure, Packaging, Git Workflow

Day 13 — Practical Python: Project Structure, Packaging, Git Workflow

Objectives

- Learn the conventional way a real Python project's files and folders are organized, so an unfamiliar codebase at a job doesn't feel disorienting
- Understand `pyproject.toml` and the basics of managing a project's dependencies (the other packages it needs to run)
- Understand what all those extra files (`__pycache__`, `.gitignore`, and so on) that show up in real projects actually are, and why they're there
- Practice the real, everyday git workflow — not just `git init` once, but the actual back-and-forth of committing and branching as you genuinely work

Why does file organization even matter?

So far, every day's exercises have lived in one single, simple folder, with just a `lesson.md`, `exercises.py`, and `solutions.py`. That's completely fine for learning exercises. But once a project grows to dozens or hundreds of files, worked on by more than one person, an organized, predictable structure becomes essential — so that anyone (including a brand-new teammate, or you yourself returning to a project six months later) can find what they're looking for by relying on well-known conventions, rather than having to explore an unfamiliar, ad-hoc mess from scratch every time.

A conventional project layout

```
my_project/
├── .venv/                # virtual environment (Day 11) -- NEVER commit this to git
├── .gitignore
├── pyproject.toml        # project metadata + dependencies (explained below)
├── README.md
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── utils.py
│   └── tests/
│       ├── __init__.py
│       └── test_utils.py
```

This particular arrangement — putting your actual package inside a folder named `src/`, rather than directly at the very top level of the project — is commonly called the "**src layout**," and it's worth knowing both what it looks like and *why* experienced Python developers deliberately choose it. It might seem like unnecessary

extra nesting at first glance, but it forces you to properly `pip install` your own package (even in a special "editable" development mode, explained below) in order to import it at all — this catches an entire, surprisingly common category of bug ("it works, but only because I happened to be running it from exactly the right folder") before it ever has a chance to reach anyone else, including a new contributor to the project who might run things from a slightly different location.

`.pyproject.toml` — the modern standard for describing a project and what it needs

```
[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.28",
]

[project.optional-dependencies]
dev = ["pytest>=7.0"]

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

This single file (`.toml` is a plain-text configuration file format, not Python code itself) replaces what, in older Python projects you might still encounter, used to be spread awkwardly across several separate files (`setup.py`, `setup.cfg`, and a plain `requirements.txt`). You'll still see those older files in some existing, legacy codebases — recognize them when you see them, but write any brand-new project using `pyproject.toml`. The `dependencies` list names everything your code genuinely needs *in order to run at all*; `optional-dependencies.dev` separately lists things only needed while *developing* the project itself (like `pytest` for testing) — someone who just wants to *use* your finished package doesn't need testing tools pulled in alongside it for no reason.

Once you have a `pyproject.toml`, the standard way to install your own project while you're actively working on it is "editable mode" — this means any change you make to your source files takes effect immediately the next time you run your code, without needing to reinstall anything in between:

```
pip install -e .
```

`.gitignore` — telling git which files it should never track

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
*.egg-info/
.env
```

A `.gitignore` file tells git (the version control tool you'll actually use momentarily) which files and folders to simply ignore entirely — never track them, never suggest committing them. Here's what each of the above

actually is, and why it belongs on this list:

- `__pycache__`/ and `**.pyc` — when Python runs your code, it automatically compiles your human-readable `.py` source files into a lower-level, faster format called "bytecode," and caches (saves) that compiled version here so future runs can start slightly faster. It's entirely a **derived**, automatically-regenerated artifact — nobody hand-writes it, and it gets recreated automatically whenever needed. Committing it to git would be pure clutter, and could even occasionally cause confusing bugs for a teammate if a stale cached copy somehow lingered around.
- `.venv`/ — your virtual environment (Day 11) is specific to **your** particular computer, and can also be quite large. Your `pyproject.toml` (or `requirements.txt`) is the actual, portable source of truth that lets any teammate recreate their own separate, independent virtual environment on their own machine.
- `.env` — a common convention for a file holding secret values (passwords, private API keys, and similar) meant only for your own local setup. Accidentally committing genuine secrets to git is a real, well-documented category of security incident that happens to real companies — and critically, once something has been pushed to a shared git repository, you generally have to treat it as permanently compromised, even if you delete it again in a later commit, since git deliberately keeps a full history of everything that was ever committed.

The real git workflow, applied to code you've actually written this week

You've been creating files all week without any version control at all. Here's the minimum realistic workflow, assuming you have git installed on your computer:

```
git init                                # run ONCE, at the very start of a project
git add .                               # stage every current file, getting it ready to be committed
git commit -m "Initial commit: week 1 exercises" # actually save that staged snapshot, with a message

# ... later, after you've changed some files ...
git status                             # see, in plain terms, what's changed since your last commit
git diff                               # see the EXACT lines that changed, added or removed
git add day12-testing-debugging/        # stage just this particular folder's changes
git commit -m "Add pytest tests for calculator module"

git log --oneline                       # see a compact, readable history of every past commit
```

****A commit message should explain **why** a change was made, not simply restate **what** changed**** — `git diff` already shows you precisely **what** changed, in full detail; a message like `"Fix off-by-one error in complete_task"` will be far more useful to you six months from now than a vague `"update tasks.py"`, especially once you're looking back trying to understand a decision you no longer remember making.

Branches are used at essentially every real programming job, for anything beyond the smallest, most trivial one-line fix:

```
git checkout -b feature/add-remove-command # create AND switch to a brand-new branch, named for the feature
# ... make your changes, commit them normally on this branch ...
git push -u origin feature/add-remove-command # upload this branch to a shared, remote location
# from there, you'd open a "pull request" on GitHub/GitLab, invite a teammate to review it, and merge it back
```

A **branch** is essentially an independent, parallel line of work, separate from your project's main line of history — it lets you make and commit changes freely without affecting the main, shared version of the code until you (and, at a real job, typically at least one reviewing teammate) are confident it's ready. The core discipline worth adopting now, even while you're still learning solo: **avoid committing changes directly onto the `main` (or `master`) branch on any team project**. Working through a branch, plus a "pull request" for someone else to review before merging, both catches bugs earlier (a second pair of eyes, before the change reaches everyone else) and leaves a genuinely useful paper trail behind — months later, `git blame` (a command that shows you exactly which commit last changed any given line) can lead you straight back to the original pull request and its description, which is often exactly the context you need to safely understand or change that code later.

Automated formatting and linting — an automated "style reviewer" for your code

Two tools you'll see wired into the setup of almost every real Python project's workflow:

- **black** — an intentionally opinionated auto-formatter. You simply run it, and it rewrites your code's formatting (spacing, line length, quote style, and more) to one single, consistent style across the entire project — permanently ending any team debate about exactly how code "should" be formatted, since the tool just decides for everyone.
- **ruff** (increasingly the modern standard, and a common, faster replacement for older tools like `flake8/pylint` that you might still encounter in some existing projects) — a fast **linter**, meaning it scans your code for likely bugs and bad patterns without actually running it: unused imports you forgot to remove, references to variables that were never defined, and other common mistake patterns.

```
pip install ruff black
ruff check .          # scan the whole project for problems
black .              # automatically reformat the whole project to one consistent style
```

At a real job, tools like these are usually run automatically as part of your **CI** ("Continuous Integration" — an automated system that runs checks on your code every time you push it), and often even earlier than that, via something called a "pre-commit hook" that runs automatically the moment you try to make a commit at all, before it's even allowed to complete. Getting comfortable running these tools yourself, locally, before you ever push your code, saves you the frustrating cycle of pushing code, waiting for CI, and then discovering it failed over a purely cosmetic formatting detail you could have caught yourself in seconds.

Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one. Working directly inside this `day13-practical-python` folder:

- 1 Create a `src/greeter/` package containing an `__init__.py` (this file can be completely empty — its mere presence is what tells Python "treat this folder as an importable package") and a `core.py` file containing a function `greet(name: str) -> str` that returns `f"Hello, {name}!"`.
- 2 Write a `pyproject.toml` for this small package, naming it `greeter`, at version `0.1.0`.
- 3 Write `tests/test_core.py` containing a `pytest` test for `greet`.
- 4 Write a `.gitignore` covering `.venv/`, `__pycache__`, `*.pyc`, and `.pytest_cache/`.

- 5 If you have git installed: run `git init` right here in this folder, then make two genuinely separate commits — one for the package plus `pyproject.toml`, and a second, later one for the tests plus `.gitignore` — each with a commit message that explains **why**, not merely **what**.
- 6 Run `pip install -e .` from this folder, then confirm that `python -c "from greeter.core import greet; print(greet('Ana'))"` works correctly from anywhere on your computer (not just from inside this specific folder), as long as your virtual environment is still active.

A fully worked reference is provided in [solution/](#) if you get stuck on the packaging mechanics specifically — but make a genuine effort to run the actual git commands yourself; there's no substitute for typing them out and seeing the real output for yourself.

Day 14 - Capstone Project: Expense Tracker

Day 14 — Capstone Project: Expense Tracker

Objectives

Build a complete, small application using nearly everything from the last 13 days, mostly unassisted:

- OOP (Days 8-9): classes for `Expense` and `ExpenseTracker`, dunder methods
- Generators/decorators (Day 10): a generator for filtering, a decorator for logging or timing
- File/data handling (Days 5, 11): JSON persistence, CSV import/export
- Errors (Day 6): custom exceptions, clean handling of bad input
- Testing (Day 12): a real pytest test suite, written by you, not given to you
- Structure (Day 13): proper `pyproject.toml` / src-layout / `.gitignore`

This is intentionally the least hand-held day. You get a specification (a written description of what the finished thing needs to do), not a scaffold (partially-written starter code). Expect to get stuck — that's the design, not a bug in the lesson. Thirteen days ago you didn't know what a variable was; today's exercise is deliberately close to "here's a real, if small, professional software task — go build it," because that's genuinely the point of the whole two weeks: proving to yourself that you can take a plain-English description of a program and turn it into working, tested code without someone walking you through each individual line.

The brief

Build a CLI expense tracker, `expense_tracker`, that:

- 1 **Adds expenses:** `python -m expense_tracker add 42.50 "Groceries" --category food`
 - Each expense has: amount (float), description (str), category (str, defaults to `"general"`), and a timestamp (use `datetime.now()` when created)
- 1 **Lists expenses**, optionally filtered by category: `python -m expense_tracker list [--category food]`
- 2 **Shows a summary:** `python -m expense_tracker summary` — total spent, and a breakdown of total per category
- 3 **Exports to CSV:** `python -m expense_tracker export expenses.csv`
- 4 **Persists between runs** as JSON (same idea as Day 7, now inside a proper class rather than free functions)

Required design elements (this is the point of the exercise — don't skip these to save time)

- An `Expense` class (Day 8-9) with a `__repr__` and, ideally, comparison support so you can sort expenses by amount or date.

- An `ExpenseTracker` class that owns a list of `Expense` objects and exposes methods like `add_expense`, `filter_by_category` (make this a **generator method**, yielding matches lazily — Day 10), `total`, `total_by_category`.
- A custom exception, e.g. `InvalidExpenseError`, raised when amount is negative or description is empty — caught at the CLI layer and shown as a clean message, never a raw traceback.
- A decorator (Day 10) applied to at least one method — e.g. logging every call to `add_expense` with a timestamp and arguments, using your own `@log_call` decorator (don't just use a library for this one — build it, to prove you understand the mechanism).
- Type hints (Day 12) on every function/method signature.
- A real `tests/` directory with pytest tests covering: adding a valid expense, adding an invalid expense (raises), filtering by category, total calculations, and CSV export producing a file with the right rows.
- Proper project structure (Day 13): `pyproject.toml`, `src/expense_tracker/`, `.gitignore`, and if you have git, real commits with meaningful messages as you go (don't do it all in one commit at the end — that defeats the point of the discipline you practiced Day 13).

Suggested (not mandatory) file layout

```

day14-capstone/
├── pyproject.toml
├── .gitignore
├── src/
│   └── expense_tracker/
│       ├── __init__.py
│       ├── models.py           # Expense, InvalidExpenseError
│       ├── tracker.py         # ExpenseTracker, log_call decorator
│       ├── storage.py         # load/save JSON, export CSV
│       └── __main__.py        # CLI entry point (so `python -m expense_tracker` works)
└── tests/
    ├── test_models.py
    └── test_tracker.py

```

`__main__.py` is what makes `python -m expense_tracker ...` work — Python looks for that specific filename when you run a package with `-m`.

How to approach 2+ hours of mostly-unassisted work without stalling out

- 1 **Write the `Expense` class first, in isolation, with a quick manual test in a REPL** (`python` interactive shell) before touching the CLI or storage at all. Confirm it behaves before building on top of it.
- 2 **Build `ExpenseTracker` next**, still testing manually in the REPL — add a few expenses, call `.total()`, before wiring up JSON persistence.
- 3 **Add JSON save/load**, confirm round-tripping works (this is where you'll hit the "how do I turn an `Expense` object into JSON and back" problem — you'll need to convert to/from a dict yourself; JSON doesn't know about your custom classes).
- 4 **Only then** wire up the CLI (`sys.argv` parsing like Day 7, or `argparse` if you want to explore something new — either is fine here).

- 5 **Write tests as you go**, not all at the end — write a test for `Expense` right after you write the class, not after the whole project is "done."
- 6 If you get stuck for more than ~15 minutes on one specific thing, that's a reasonable point to peek at the relevant part of `solution/` for just that piece — not the whole file.

A hint on the JSON-with-custom-objects problem, since it's genuinely new today

You haven't been shown this pattern yet on purpose — figuring it out is part of the exercise — but here's a nudge: `json.dump/json.load` only understand the built-in types from Day 11's table. To persist an `Expense`, you need to convert it to a plain dict before dumping, and rebuild an `Expense` from a dict after loading:

```
def to_dict(self) -> dict:
    return {"amount": self.amount, "description": self.description, ...}

@classmethod
def from_dict(cls, data: dict) -> "Expense":
    return cls(data["amount"], data["description"], ...)
```

`@classmethod` is a decorator you haven't formally covered — it marks a method that receives the `*class*` (conventionally named `cls`) instead of an instance as its first argument, making it useful as an alternate constructor. Look up `@classmethod` and `@staticmethod` briefly today; both are common enough in real code that you should recognize them, even though today is the first time you're using one for real.

Definition of done

- All 5 CLI commands work end to end, tested manually.
- `pytest` passes with no failures, and covers the required cases listed above.
- No raw tracebacks reach the user for the invalid-input cases you're asked to handle.
- Project installs with `pip install -e .` and runs via `python -m expense_tracker`.

Reference solution

`solution/` has a complete working implementation. Give yourself a genuine attempt — ideally the full 2 hours — before opening it. The value of this whole two weeks culminates here: if you can build this mostly on your own, you're at the level this track promised.